The Journal of Systems and Software 230 (2025) 112542

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

In practice

Smart manufacturing: MLOps-enabled event-driven architecture for enhanced control in steel production☆

Bestoun S. Ahmed [a,f,*], Tommaso Azzalin [b], Andreas Kassler [a,c], Andreas Thore [d], Hans Lindbäck [e]

[a] Karlstad University, Computer Science Department, Karlstad, 65635, Sweden
[b] University of Bologna, Department of Computer Science and Engineering, Bologna, 40126, Italy
[c] Deggendorf Institute of Technology, Faculty of Applied Computer Science, Deggendorf, 94469, Germany
[d] Smart Industrial Automation, RISE Research Institutes of Sweden, Västerås, Sweden
[e] Bharat Forge Kilsta AB, Karlskoga, Sweden
[f] American University of Bahrain, Riffa, Bahrain

ARTICLE INFO

Keywords:
Steel production
Advanced machine learning (ML)
Deep reinforcement learning (DRL)
MLOps-driven architecture

ABSTRACT

We explore a Digital Twin-Based Approach for Smart Manufacturing to improve Sustainability, Efficiency, and Cost-Effectiveness for a steel production plant. Our system is based on a micro-service edge-compute platform that ingests real-time sensor data from the process into a digital twin over a converged network infrastructure. We implement agile machine learning-based control loops in the digital twin to optimize induction furnace heating, enhance operational quality, and reduce process waste. Key to our approach is a Deep Reinforcement learning-based agent used in our machine learning operation (MLOps) driven system to autonomously correlate the system state with its digital twin to identify correction actions that aim to optimize power settings for the plant. We present the theoretical basis, architectural details, and practical implications of our approach to reduce manufacturing waste and increase production quality. We design the system for flexibility so that our scalable event-driven architecture can be adapted to various industrial applications. With this research, we propose a pivotal step towards the transformation of traditional processes into intelligent systems, aligning with sustainability goals and emphasizing the role of MLOps in shaping the future of data-driven manufacturing.

1. Introduction

During the past two decades, many companies have faced production problems that have led to low efficiency, poor product quality, and high costs. In particular, traditional and heavy industries, including steel and metal processing, also face material waste challenges that have a significant environmental impact and reduce their profits. An essential reason for product defects and high cost is the low degree of automation and digitalization of existing equipment used on the shop floor (Cagle et al., 2020). For example, in a forging shop, some of the main processes, such as heating steel bars, still depend on manual adjustment by operators who have been trained for several years. Here, poor process control can lead to material degradation due to production temperature falling outside the specification, and product quality cannot be automatically detected during production.

Although recent advances towards the fourth industrial revolution (Industry 4.0) (Qin et al., 2016) have prompted many companies to improve productivity and increase the reliability of equipment and production lines through a greater degree of automation and digitalization, significant challenges remain to implement these technologies effectively in complex industrial environments. This paper addresses the specific scientific challenge of developing a robust and adaptive control system for steel forging processes that can operate in real-time under dynamic conditions, a problem that existing approaches have not fully resolved.

Traditional approaches to process control in steel manufacturing have typically relied on conventional feedback control systems, which often struggle with the complex, non-linear dynamics of industrial heating processes (Ferreiro et al., 2016; Wang et al., 2018). Recent studies have highlighted the limitations of these conventional methods, particularly their inability to adapt to varying production conditions and material properties (Wang and Xu, 2011; Kwak and Park, 2011; Chai et al., 2017).

☆ Editor: Dr. Marcos Kalinowski.

* Corresponding author.
E-mail address: bestoon82@gmail.com (B.S. Ahmed).

https://doi.org/10.1016/j.jss.2025.112542
Received 28 February 2024; Received in revised form 1 March 2025; Accepted 17 June 2025
Available online 7 July 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

As one of the emerging technologies, digital twins (DT) are becoming increasingly important in improving digitalization in the process industry due to rapidly evolving simulation and modeling capabilities (Perno et al., 2022), also leveraging the vast amount of compute processing from edge/cloud infrastructures. A DT is a high-fidelity virtual model aimed at emulating physical processes and behaviors with the ability to evaluate, optimize and predict (Graessler and Poehler, 2017). It also provides a real-time link between the physical and digital worlds, allowing companies to have a complete digital footprint of their products and production processes throughout the production cycle (Khan et al., 2018). In turn, companies can realize significant value in areas of improved operations, reduced defects, increased efficiency, and enabled predictive maintenance (Panagou et al., 2022). With the acquisition of real-time data, a DT can help operators understand the actual production process and make preventive decisions when an anomaly occurs (Liu et al., 2021). However, the integration of DTs with advanced control algorithms in industrial settings presents significant scientific and technical challenges, particularly in terms of real-time data processing, model accuracy, and system adaptability.

While intelligent algorithms can be used together with DTs to control real systems, such as optimizing energy consumption, reducing waste, and improving product quality, their effective implementation in steel production environments remains an open problem. Traditional control algorithms in steel production typically involve control theory, mathematical modeling, and statistical analysis (Pimenov et al., 2022; Pata et al., 2023). Although these methods have yielded notable improvements, they often struggle to adapt to the dynamic and complex nature of industrial environments. Recently, data-driven algorithms using Machine Learning (ML) have demonstrated their potential to solve complex dynamic control problems in various domains, from robotics to autonomous vehicles (Wang and Chortos, 2022; Elallid et al., 2022). However, their application in optimizing industrial processes, particularly in steel production, remains largely underexplored and presents unique scientific challenges. These challenges include: (1) the formulation of appropriate data-driven algorithms that can handle the high-dimensional, non-linear dynamics of steel production processes; (2) the development of robust learning frameworks that can operate reliably in noisy, industrial environments; and (3) the integration of these advanced algorithms with existing industrial control systems.

Furthermore, the operationalization of these algorithms poses substantial challenges due to complex system architecture requirements, necessitating a detailed design process. The deployment of such algorithms requires careful attention and involves the resolution of various challenges, thereby requiring the consideration of multiple architectural decisions from the perspective of Machine Learning Operations (MLOps). It is worth mentioning that the adoption of standard, commercialized cloud solutions from providers is rendered challenging in applications like this due to security and the high latency involved. As we aim to control industrial processes, sophisticated edge-compute architecture is required to minimize the control latency. This requires developing and implementing a custom architecture tailored to address the unique demands of controlling the production process.

In this paper, we propose a novel technical solution that addresses these scientific challenges by embedding a DT of the production process into an edge compute infrastructure to optimize the control of a steel forging heating furnace. Our solution leverages a micro-services-based architecture for integrating MLOps pipelines that receive and process sensorial data from PLCs. The core scientific contribution lies in the development of a novel Deep Reinforcement Learning (DRL) framework that can learn and adapt to the complex dynamics of the steel forging process in real-time, a problem that existing approaches have not effectively solved. To this end, our work makes the following contributions:

- Advanced Integration of DRL and Industry 4.0 Principles: We present a novel framework that seamlessly combines advanced DRL techniques with Industry 4.0 principles, addressing the scientific challenge of creating adaptive, real-time control systems for complex industrial processes.
- Innovative MLOps-Driven Architecture: We develop a unique MLOps-driven architecture that overcomes the challenges of implementing and maintaining ML models in industrial settings, providing a scientifically grounded approach to ensuring robustness and efficiency in production environments.
- Novel DRL Approach for Induction Furnace Heating: We introduce a new DRL algorithm specifically designed to control electrical power during the heating process of the induction furnace, addressing the scientific challenge of optimizing non-linear, high-dimensional industrial processes in real-time.
- Comprehensive Empirical Validation: We provide extensive empirical results that validate the scientific and practical benefits of our system, demonstrating significant waste reduction and operational quality improvements in a real industrial setting.

The remainder of this paper is organized as follows. Section 2 details the background of the furnace and the operational process, illustrates the problem statement, and provides a comprehensive review of related work. Section 3 presents the MLOps-driven system architecture for digital twin-based control, outlining the key aspects of our approach. Section 4 delves into the DRL framework for digital twin-based process control, explaining the necessary background and implementation details. Section 5 describes the deployment process, including model maintenance and management updates crucial in MLOps. Section 6 presents our experimental evaluation, demonstrating the effectiveness of our approach through comprehensive empirical results. Section 7 discusses the impact of our system on sustainability, efficiency, and cost-effectiveness, including a detailed use case scenario. Finally, Section 8 concludes the paper and outlines directions for future work.

2. Background and related work

In this section, we briefly introduce the necessary background on the furnace and the operational process that we aim to control from the architecture that we are proposing. We detail the problems that arise when semi-manually controlling the furnace operation. We also summarize related work and how our work relates to it.

2.1. Induction heating for the forging shop

This research was conducted in collaboration with Bharat Forge Kilsta AB, a leading manufacturer in the forging industry located in Karlskoga, Sweden. Bharat Forge Kilsta specializes in producing high-quality forged and machined components for the heavy vehicle and industrial sectors. The company's production facility includes state-of-the-art forging equipment, including the induction heating furnace, which is the focus of this study.

In our work, we aim to automate the control of a furnace that is used to heat steel in a forging shop in a production plant. Steel in the plant comes in the form of rods of different lengths, diameters, and physical properties. The furnace heating these rods is built as a production line of induction coils that are approximately 1 m long and separated by a gap of about 25 cm. The induction coils are grouped into five zones of four coils each. This means that the furnace is approximately 25 m long. Fig. 1 shows a simplified graphical representation of the furnace and its zones.

As shown in Fig. 1, rods that need to be inserted inside the furnace are first loaded into a warehouse located before the starting point of the production line and its dimensions are not included in the 25 m range. Roller conveyors between induction coils allow rods to move

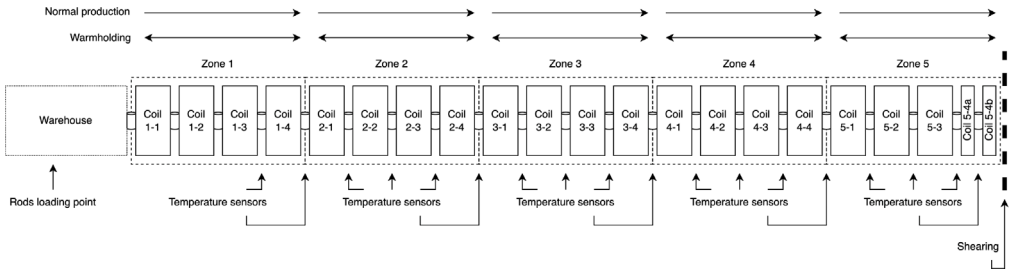

Fig. 1. A graphical and simplified representation of the furnace.

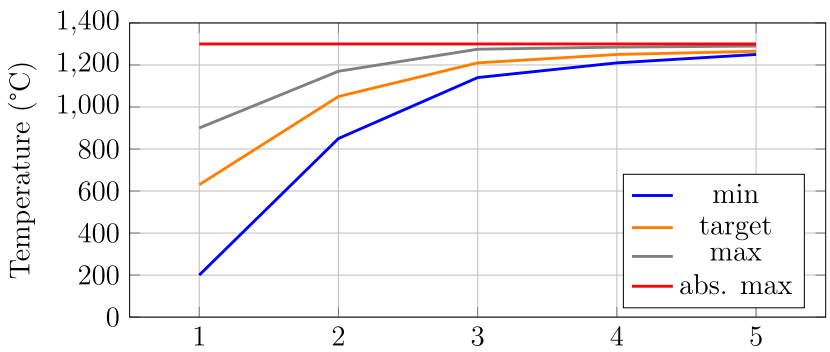

Fig. 2. Temperature ranges in the furnace.

back and forth for heating. The fifth zone has five induction coils. The last induction coil is split into two halves. This splitting allows an extra roller to be placed between the coils to speed up the rod before reaching the end of the furnace, where it is sheared and produces billets of the desired length that can then go to the forging part of the plant.

The induction coils are heated through electrical power that human operators can control. The heat is induced and created into the steel by induction from the coils. All five zones have different powers to allow the rod to reach the appropriate target temperature when approaching the end of each of them. Fig. 2 shows the minimum, maximum, and target temperatures of all five zones. The current specifications require the rod to be heated intensively in the first two zones, with even a major intensity in the second one, and then progressively reduce the power in the last three zones to reach the desired temperature on the steel rods without too many variations. To check what is happening in the furnace and to act accordingly, several sensors (pyrometers) measure the temperature that is placed at every gap between the induction coils and after shearing, some to obtain the rod positions and the velocity at which they are moving, and finally measure the power which is applied to the induction coils. Pyrometer sensors are placed to obtain quick feedback from the production line and allow human operators to act based on the retrieved values. The furnace can work in two modes that are designed for different situations:

- **Normal production**, in which the rods are loaded from the warehouse inside the furnace and moved forward with constant speed until they reach a roller near the end of the furnace. At this point, the speed is increased and the bar is moved out of the furnace and to the shear. The electrical power applied to the induction coils is very high and follows recipes designed by the company on the basis of the diameter and material of which the rods are made. The recipes are known to work well in most cases and guide the PLC in setting the proper power for the coils in the different zones. Human operators can apply small manual adjustments to power settings based on their long experience. This straightforward mode of operation is easily manageable by human operators.
- **Warmholding**, in which no new rods are loaded from the warehouse. Instead, the rods inside the furnace move back and forth and are heated to a much lower temperature to avoid overheating. The goal is to allow them to go through shearing and later forging at the right temperature. There are different reasons for switching to warmholding, including furnace problems that require manual intervention. The problem is that the induction coils are separated by gaps in which the rods cool and lose heat. Therefore, different rods and parts of the same rod may have different temperatures. While back-and-forth can preserve the rods from underheating, it produces another problem called zebra temperature patterns, in which a rod has both parts at a high and a low temperature. If these patterns are too difficult to handle, operators could decide to stop the process, unloading the rods, wait for them to cool down, and start over, which would waste a lot of time and energy.

Scrapping occurs due to large variations in the steel rods' forging temperature, mainly when the industrial plant is restarted after downtime or during warmholding mode. The rods inside the furnace that are already partially heated need to maintain their temperature without being damaged or losing heat, as this would mean a great loss of time and energy. The forging temperature window is very narrow, and having material that is under- or overheated leads to poor-quality products. Human operators control the forging process and can easily keep the steel within the temperature window during the normal production scenario, but struggle when the rod should be kept in the forging system for a certain time with a limited temperature range during maintenance and warmholding mode.

2.2. Related work

Efforts to improve the efficiency, sustainability, and cost effectiveness of steel production have been the subject of extensive research. Optimizing steel production processes has long been challenging due to its significant impact on industrial sectors and the environment. The complexity of steel manufacturing, which involves various stages of intensive energy, requires continuous innovation to improve operational efficiency while minimizing environmental impact. He and

Wang (2017) provided a comprehensive review of current practices and challenges in optimizing energy efficiency within the steel industry, identifying key areas such as energy recovery, process optimization, and waste minimization. Their work underscores the importance of energy optimization in reducing operational costs and as a critical component in achieving broader sustainability objectives. Building on this foundation, Zhang et al. (2018) conducted an in-depth analysis of energy consumption patterns in integrated steel plants, proposing a multi-objective optimization model that balances energy efficiency, production costs, and environmental impact. Their research highlights the intricate relationships between various production parameters and their collective influence on overall plant performance. In search of sustainability, Pardo and Moya (2013) explored the potential of breakthrough technologies in the steel industry to significantly reduce CO2 emissions. Their study evaluates emerging technologies such as hydrogen-based direct reduction and carbon capture and storage, providing insights into the future direction of sustainable steel production.

In addition to energy efficiency, improving operational quality and process control in steel production has garnered considerable attention. Kano and Yoshiaki explored methods to improve operational quality in steel manufacturing through advanced process control techniques (Kano and Nakagawa, 2008). Their study underscores the relevance of optimizing production processes for quality improvement and resource efficiency, aligning with our objectives. In another effort, Zanoli, Cocchioni, and Pepe (Zanoli et al., 2018) introduced a two-layer predictive control strategy based on linear models with adapted online horizons. This approach is applied within an advanced process control framework to optimize the reheating furnace of pusher-type billets in an Italian steel plant. The synthesized control system successfully replaced manual control actions, significantly improving process control, energy efficiency, and the fulfillment of product quality specifications. Further advancing process control, Branca et al. (2020) investigated the current and future aspects of the digital transformation in the European steel industry. Their research demonstrates how digital transformation and smart manufacturing principles can revolutionize traditional steel production processes, leading to enhanced productivity and quality control.

DRL application in solving complex control problems has gained prominence in various domains. Mnih et al. (2015) demonstrated the transformative potential of DRL by achieving human-level control in gaming environments. This landmark work underscores the adaptability and power of DRL in mastering intricate tasks, setting the stage for its exploration in industrial contexts. However, in industrial settings, the application of DRL is still evolving. Recent research has shown promising results. For example, Huang et al. (2019) applied DRL to optimize the scheduling of steel production processes, resulting in significant improvements in energy efficiency and production quality. Their work illustrates the growing interest in leveraging DRL for steel production optimization, mirroring our approach. Most recently, Ferreira Neto et al. (2024) made significant achievements applying DRL to maintenance optimization in steel production. Their study proposes a DRL-based optimization approach for determining a scrap-based steel production line's optimal inspection and maintenance planning. The research considered practical aspects such as the uncertainty of maintenance duration and variable production rates, demonstrating the adaptability of DRL to complex, real-world industrial scenarios. Their findings indicate the potential for significant financial savings, highlighting DRL's power to enhance industrial competitiveness in the steel sector.

Although the incorporation of MLOps into industrial processes is an emerging and influential trend, its application remains relatively underexplored in heavy industries such as steel production. A growing body of technical reports highlights the importance and utility of MLOps in industrial contexts. However, comprehensive studies that showcase the results and lessons learned from implementing customized MLOps solutions are notably lacking. In particular, Zhao et al. (2022) recently delved into the implementation of MLOps in the manufacturing sector, shedding light on its key role in ensuring the resilience and scalability of machine learning solutions within production environments. Their findings underscore the criticality of MLOps in effectively facilitating the operationalization of advanced ML techniques, aligning with the core objectives of our paper. However, despite these valuable insights, there remains a gap in the literature in presenting concrete results and experiences derived from applying customized MLOps solutions in heavy industries such as steel production. Building on this, Tao et al. (2019) proposed a framework for integrating the concepts of MLOps with DT technology in smart manufacturing. Their research demonstrates how these concepts can enhance the accuracy and reliability of digital twin models, leading to more effective decision-making and process optimization in complex industrial settings like steel production.

Despite advances in applying ML and DRL to steel production optimization, significant challenges remain to implement and maintain these solutions in real-world industrial settings. Integrating MLOps and EDA in smart manufacturing presents both opportunities and hurdles, particularly in the steel industry. Faubel, Schmid, and Eichelberger (Faubel et al., 2023) highlighted the critical role of MLOps in ensuring the resilience and scalability of ML solutions within production environments. However, applying MLOps in heavy industries such as steel production is still in its infancy. The challenge lies in adapting MLOps practices, typically used in software development, to the unique requirements of steel manufacturing processes. EDA offers the potential for real-time responsiveness in complex manufacturing environments. As noted by Theorin et al. (2017), EDAs can significantly enhance the flexibility and adaptability of control systems in industrial settings. However, integrating EDAs with existing steel production infrastructure, which often relies on legacy systems, presents considerable technical challenges. Combining MLOps and EDA in steel production control systems introduces new complexities in data management, model deployment, and system maintenance. As Tao et al. (2019) observed in their work on digital twins in smart manufacturing, the sheer volume and variety of data generated in modern steel plants require sophisticated data processing and analysis pipelines. Ensuring the reliability and performance of these pipelines in a 24/7 production environment is a significant challenge.

3. MLOps-driven system architecture for digital twin-based control

To automate the production process, reduce material waste, and improve production quality, we aim to develop AI-based software that can automatically control the power adjustments in the furnace induction coils based on data collected from the plant in real-time by the many different sensors.

Our approach integrates three key components that will be discussed throughout this paper: (1) a Digital Twin that serves as a virtual model of the physical furnace, (2) an EDMA that enables real-time data processing and control, and (3) DRL algorithms that optimize decision making. In this section, we focus on the EDMA component, which forms the foundation for implementing and operating our DT. Later sections will demonstrate how this architecture supports the DRL algorithms that drive intelligent control decisions within the Digital Twin environment.

To meet the stringent security requirements of the company, traditional off-the-shelf cloud solutions are not a feasible option. As temperature power adjustments need to be applied quickly due to the movement of the rod and the potential for overheating, an edge-computing approach is required. This requires a custom architecture framework to effectively manage and control production operations. To initiate this process, it is critical to establish a baseline production mode that can be automatically managed, potentially leveraging AI

techniques, to ensure the system's correctness and efficiency. This operation state will be called the "proper normal production mode" throughout this paper.

For both normal production and warmholding modes, a robust operational infrastructure is required to facilitate AI-driven control algorithms in real-time production scenarios. While the AI algorithms that govern each production mode may differ according to specific conditions, the basic infrastructure remains constant. Consequently, our work focuses primarily on the development and implementation of the operational infrastructure, adopting an MLOps perspective. Our primary focus within this paper centers on automating the normal production mode. It is important to clarify that the emphasis here is not on the complexities of the algorithm, but rather on the architecture and infrastructure supporting it. Integrating a more complex algorithm into existing infrastructure follows a similar procedure, but such advanced algorithmic developments fall beyond the scope of this paper.

In our investigation of revolutionizing steel production by integrating DRL techniques, we have devised an MLOps architecture to guide our efforts. MLOps represents a structured framework necessary to manage the entire lifecycle of control software framework, including the ML solution. This framework is the backbone of our approach, which ensures seamless management of ML models throughout their lifecycle. The MLOps methodology comprises three key aspects: design, implementation, and deployment, each playing an important role in the success of our work.

In the design phase, we plan the entire transformation process. We assess the requirements for integrating DRL techniques and a DT into the steel production workflow, considering the unique challenges and complexities of this industry. We define the architecture, infrastructure, and workflow that will facilitate the seamless operation of our DRL-based system. During the implementation, the initial development of the ML model will take place, taking into account the data and the choice of the models. We also show the implementation of a robust ML pipeline that encompasses data collection, preprocessing, model training, validation, and evaluation. The heart of our implementation phase is the development of the DRL algorithm, which controls the electrical power consumption during the heating process. This phase involves the development and testing of the model to ensure the reliability and compatibility of the DRL model with the existing production infrastructure. We then show how to deploy the DRL model into the production environment and establish the infrastructure for real-time decision making. The following sections will go through these phases in detail and show the necessary parts of each phase.

The initial stage of the MLOps approach was the design phase. Here, we outline the overarching architecture and strategy to integrate DRL into steel production processes. To solve the problem described, we present the design of the system, which is composed of three parts:

- **Furnace**, which can be considered just the production line, the sensors, and the actuators of interest;
- **Architecture for data input**, which collects data from plant sensors and makes them available for processing, together with receiving input to control actuators and request actions to be made;
- **Event-driven microservices architecture**, which performs a data processing.

Fig. 3 shows a high-level overview of these three parts of the system and how they are interconnected.

3.1. Furnace

Monitoring the forging process requires many sensors collecting data and a proper refresh interval to always have an up-to-date plant overview. Furthermore, it requires an adequate number of actuators for the operators to control the process's behavior. The furnace has sensors for different purposes placed in many parts to provide a complete overview of operations over time. The sensors measure the following:

- The temperature, through pyrometers that are placed in every gap between induction coils and after shearing, to check if the billets have a sufficient quality to be used or need to be scrapped;
- The position of the rods inside the furnace;
- The velocity at which the roller conveyors move the rods;
- The power applied in the induction coils.

Operators can control, through the use of actuators, the following parameters in production:

- the applied voltage in the induction coils, which defines the emitted power of the induction coils;
- the speed at which the roller conveyors turn, which defines the velocity at which the rods move;
- whether to let the furnace work in normal production or warmholding mode;
- whether to enable the automatic control or not.

3.2. Architecture for data input

All sensors and actuators are connected and controlled by Siemens 1500 PLCs (Programmable Logic Controllers). PLCs are ruggedized computer systems designed for industrial automation and control, capable of withstanding harsh industrial environments. The software of the PLCs is, in turn, connected to an instance of an OPC-UA[1] server, which acts as a middleware layer between the data sources and the consumers. Within the OPC-UA server, the data are stored as strongly typed key–value pairs, called tags, which can be set to be read/written by OPC-UA clients. In addition, the PLCs receive data from the sensors and the OPC-UA server updates the values of the connected tags. Here, sanity checks can be performed by updating the values of the tags through code written for the PLCs. An OPC-UA server can work in two ways. In this paper, we use both methods in different situations:

- *client–server* or *pull-based*, in which the tags' values are requested from the server by the clients;
- *publisher-subscriber* in which clients subscribe and receive updates every time a tag value is updated.

Instead of constantly accessing the OPC-UA server, tag updates are pushed on a topic/channel of a message broker (that is, Telemetry), which allows other services to subscribe to it. Fig. 3 shows the connections between PLCs, OPC-UA, and the message broker.

3.3. Event-driven microservices architecture

The message broker topic contains the telemetry data coming from the OPC-UA server. This topic is the input for a software architecture that acts as a data processing pipeline, culminating in updates for the induction coils. These updates are voltages that, applied on the induction coils, would change the emitted powers. In this paper, we have selected the Event-Driven Microservices Architecture (EDMA) because the microservices have small tasks to fulfill and are designed to perform their jobs as fast as possible. Since there are many of them in the pipeline, low latency is a key requirement to avoid adding time overhead at every step. The *event-driven* means that the data are processed mainly in a publisher/subscriber fashion (with a message broker acting as a middleware). The data that are pushed on the topics of the message broker is regarded as an event that must be processed. To this end, the data processing pipeline has the following steps:

- The telemetry data gets parsed and reformatted to get stored and allow further analysis;

[1] https://opcfoundation.org/about/opc-technologies/opc-ua/.

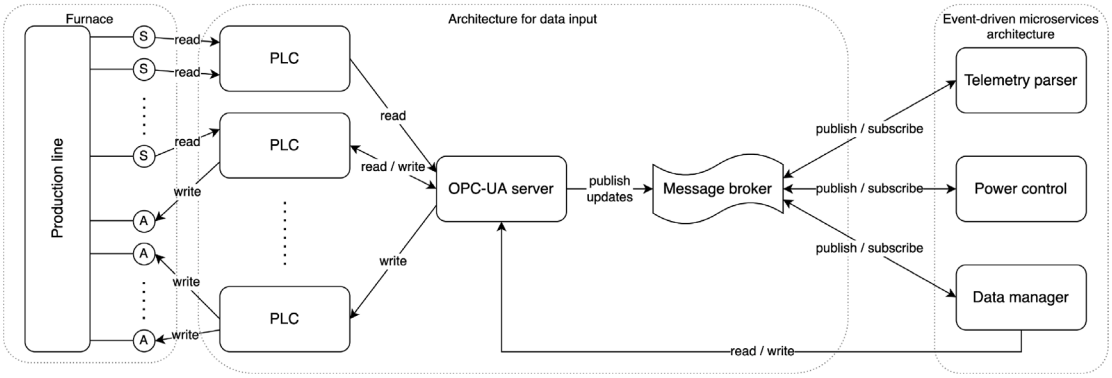

Fig. 3. System architectural overview.

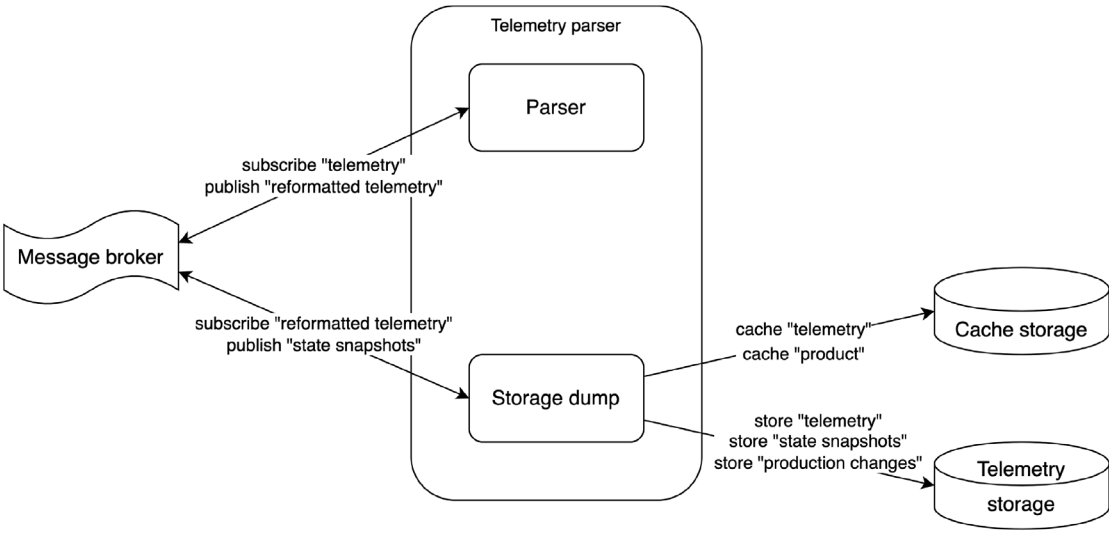

Fig. 4. The Telemetry parser's structure and its main interconnections.

- The results get collected over time to form complex data structures, called state snapshots, which aim to represent the furnace's state at the state's creation time. These data structures also get stored for further analysis;
- from the reformatted telemetry, some pieces of data are either stored or cached in appropriate ways to be used within the processing of the pipeline;
- The state snapshots are given as input to some power control algorithms returning power updates (V), which are the voltages to set in the induction coils of every given zone;
- The power updates undergo sanity checks, and those voltages that pass them can be written to the corresponding tags in the OPC-UA server.

As is clear from Fig. 3, there are three main building blocks of the EDMA: (1) Telemetry Parser, (2) Power Control, and (3) Data Manager. The telemetry parser parses and reformats the telemetry data, creates state snapshots and tracks the material under production. The power control gets the state snapshots and processes them with power control algorithms that decide how to update the heating by the induction coils. The data manager gets instructions on how to update the heating of the induction coils, update the corresponding tags on the OPC-UA server, cache pieces of data from the OPC-UA server, and ensure that the connection with it is active. The following subsections illustrate the necessary details of these building blocks.

3.3.1. *Telemetry parser*

The telemetry parser is the macroservice in the data processing pipeline that processes the raw telemetry coming from the OPC-UA server. Fig. 4 shows the microservices in place to perform the task of reformatting data and creating snapshots of the state.

As is clear from the structure, there are four active blocks in addition to the message broker:

- **Parser**, which gets the data from the message broker's topic containing the telemetry, reformats it, and publishes it into another topic *reformatted telemetry*;
- **Storage dump**, which is subscribed to the *reformatted telemetry* topic to store the incoming records in Telemetry storage, caches them together with data of the material under production in Cache storage, stores when the material under production changes, collects data to create state snapshots for later storing them in Telemetry storage and publishing them into a topic *state snapshots*;
- **Cache storage**, which is a storage service optimized to cache small amounts of data and has fast reading and writing times;
- **Telemetry storage**, is another storage service that has fast writing times and is optimized for memorizing structured and unstructured data.

3.3.2. *Power control*

Power control is designed to consider the two modes that the furnace has to deal with: normal production and warmholding. For this

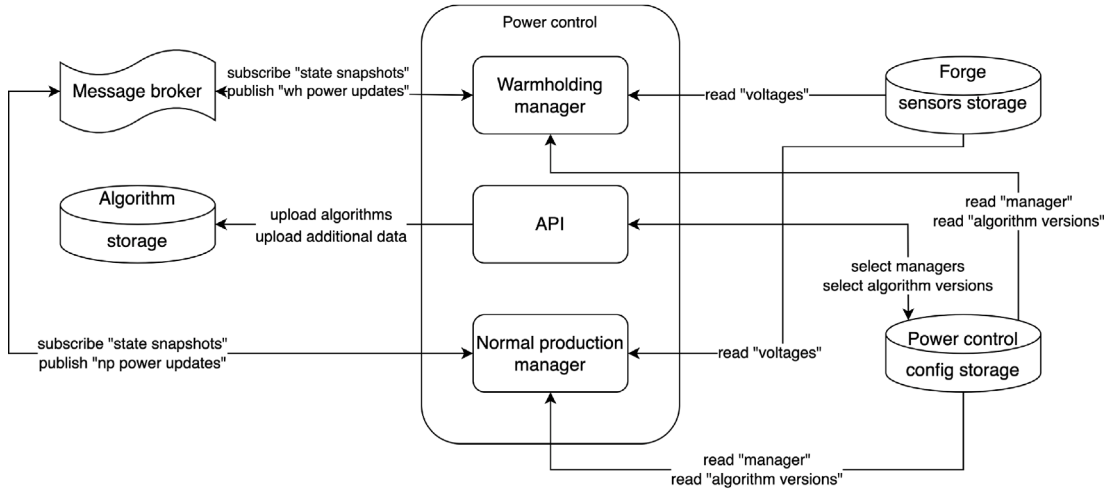

Fig. 5. The Power control structure and its main interconnections.

reason, there are two microservices called **Normal production manager** and **Warmholding manager** that determine the power updates for the two different modes at any given time. Fig. 5 shows the power control structure and the main interconnections.

Both managers subscribe to the topic state snapshots and publish the voltages for the induction coils in the topic *np power updates* and *wh power updates*. The former contains voltages applicable during normal production, the latter for warmholding. They are also updated with the current voltages applied to each zone by reading them from Forge sensors storage. Since the algorithms decide only what to do with the power (W) in the induction coils, there needs to be a way to convert from powers (W) to voltages (V). This conversion requires the current voltages for each zone, $V_{i,old} \forall i \in 1, \ldots, 5$, read from the Forge sensors storage. The conversion also requires the power in each zone $P_{i,old} \forall i \in 1, \ldots, 5$ before the algorithm makes its decision, based on the state snapshots. In addition, the new power $P_{i,new}$ after the algorithm makes its decision is required for the conversion. Based on this, the new voltage for the ith zone is computed in Eq. (1):

$$V_{i,new} = \left\lceil V_{i,old} \cdot \sqrt{\frac{P_{i,new}}{P_{i,old}}} \right\rceil \tag{1}$$

Therefore, a complete power update is computed in Eq. (2):

$$PowerUpdate = \left(V_{1,new} V_{2,new} V_{3,new} V_{4,new} V_{5,new}\right) \tag{2}$$

More than one manager should ideally be available in the architecture since the microservice API allows for manager and algorithm selection, i.e., which manager to load for both the modes and which versions of the algorithms to use, as well as for uploading of new algorithm versions to Algorithm storage when they are ready. Rebooting is not necessary since the microservice is designed with a hot-swapping concept.

3.3.3. Data manager

The data manager is a collection of microservices that have only one of them connected to the message broker. Yet, all have communication links with the OPC-UA server. Fig. 6 shows this concept. The design idea is to concentrate on all microservices required to communicate with that server. Here, the power updater is subscribed to both the np power updates and wh power updates topics and updates the tags controlling the voltages of the induction coils on the OPC-UA server by selecting them from the topic corresponding to the currently active mode. The forge data retriever periodically retrieves the current voltages, the active mode, and material under production from the OPC-UA server and caches them into the forge sensors storage. The connection check constantly reads from and writes on a tag the OPC-UA server (as heartbeat) to check connection latency between the services and the server.

4. DRL for digital twin (DT)-based process control

Building upon the EDMA foundation described in the previous section, we now focus on how DRL is integrated with our DT to enable intelligent control of the steel production process. The DT environment, supported by our event-driven architecture, provides the ideal platform for implementing and training DRL algorithms that can learn optimal control strategies.

In contrast to Supervised and Unsupervised ML, Reinforcement Learning (RL) is the approach to learning from interaction within an environment. The mathematical approach to RL is based on the theory of the Markov Decision Process (MDP), which allows describing problems of this type as tuples $\langle S, A, R, p, \gamma \rangle$ (S is the set of possible states, A is the set of performable actions, R is the reward function, p is the probability distribution that describes the next state. The reward given for the initial state and the action carried out in it γ is the so-called discount factor) (Ogunfowora and Najjaran, 2023). The learning entity in RL is called *agent*, which interacts with the *environment* to seek rewards to maximize them over time (the value to maximize is the expected cumulative reward, in which R and γ play an important role) (Sutton and Barto, 2018). Fig. 7 is a high-level representation of the RL workflow.

The agent can learn the best way to act through interaction with the environment. Taking into account a *model-free* RL problem, in which there is no access to the whole model, or it is impossible to have it, the approach to learning follows two paths:

- *policy-based* in which a policy π is sought, which tells the agent what to do (i.e., what action to take) given a state $s \in S$ and an action $a \in A(\pi(a \mid s))$ is learned;
- *value-based* in which a value function Q_π or V_π (depending on the input, $Q_\pi : S \times A \to \mathbb{R}, V_\pi : S \to \mathbb{R}$) is learned, that implicitly use π, as remarked by the subscript in Q and V.

The previous concepts of $\pi(a \mid s)$, $V(s)$ and $Q(s,a)$ are now represented by $\pi(a \mid s;\theta)$, $V(s;\theta)$ and $Q(s,a;\theta)$ in which θ are the weights of the DNN inside the agent (Li, 2017). The following sections explore one value-based method, namely Deep Q-Learning/Deep Q-Network (DQN) (Mnih et al., 2015) and one policy-based method, Proximal Policy Optimization (PPO) (Schulman et al., 2017). DQN is chosen for its straightforward nature, popularity, and notable track record of delivering good results in the literature. In contrast, PPO is included to complement the evaluation, offering a complementary perspective to improve our understanding and provide a comprehensive assessment of the proposed approaches.

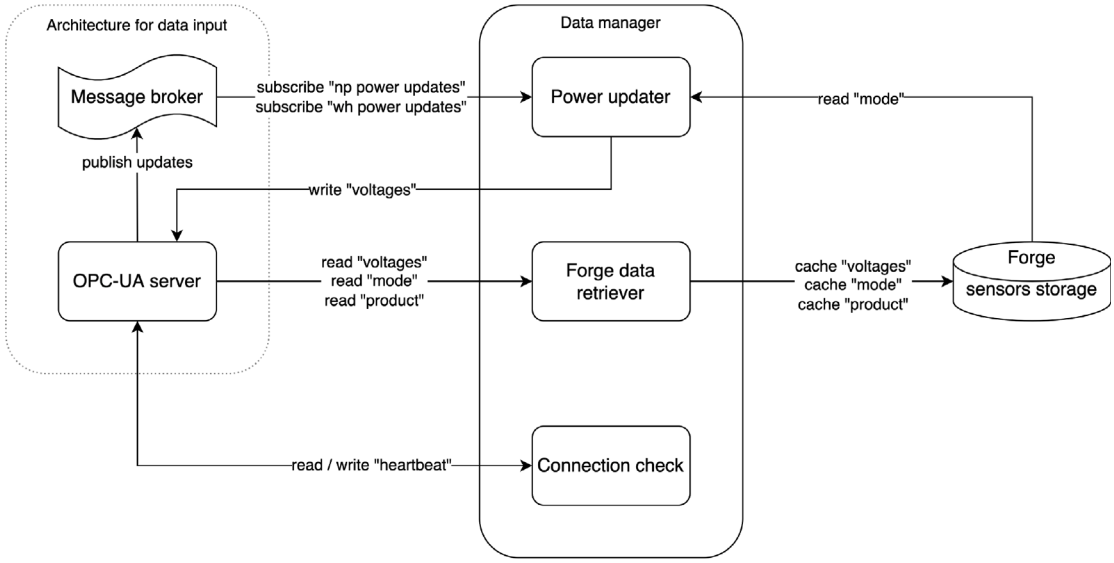

Fig. 6. The Data manager's structure and main interconnections.

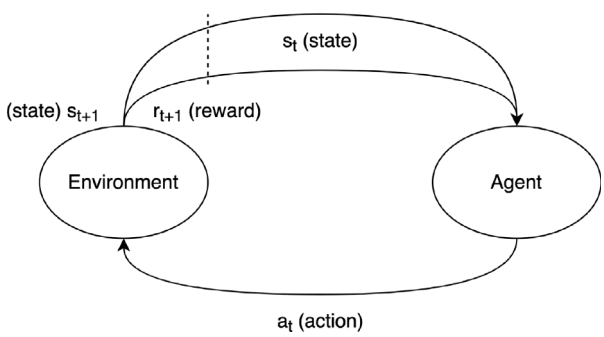

Fig. 7. A high-level representation of the RL workflow.

4.1. Deep Q-learning

An important limitation of DQN is that actions can only be discrete. The goal is to approximate the optimal value function Q as shown in Eq. (3):

$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[\sum_{i=0}^{\infty} \gamma^i r_{t+1} \mid s_t = s, a_t = a, \pi\right] \tag{3}$$

The experience replay is a memory from which the DRL agent can randomly sample the previous interactions with the environment and use them to update θ in the DNN representing the Q function. The target network is a DNN identical to the other one representing the Q function but is used to compute the expected cumulative reward. Its weights θ are not updated together with θ; instead, only every C step (C is a hyperparameter of the training process). The actions are usually chosen with a ϵ-greedy strategy to balance exploration and exploitation.

4.2. Proximal Policy Optimization (PPO)

The idea behind PPO is to try to optimize a similar algorithm, namely Trust Region Policy Optimization (TRPO), and combine it with an Actor–Critic with Experience Replay (ACER) approach. The actor network is trained to learn the policy π, while the critic network tries to learn the value function V. To learn the optimal π, the agent tries to minimize the loss:

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}\left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S\left[\pi_\theta\right]\left(s_t\right)\right] \tag{4}$$

in which:

$$L_t^{CLIP}(\theta) = \hat{\mathbb{E}}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}\left(r_t(\theta), 1-\varepsilon, 1+\varepsilon\right)\hat{A}_t\right)\right] \tag{5}$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ represents the ratio between the new and old policies at each step. The loss $L_t^{CLIP}(\theta)$, as shown, uses a combination of min and a clipping function to limit this ratio: the idea of doing so is to limit aggressive positive updates to π (to converge slower but more stable) and instead allow negative updates, when necessary. The $\hat{A}_t$ is the advantage at time t, calculated as $\hat{A}_t = \delta + \gamma \cdot \lambda \cdot m_t \cdot \hat{A}_{t+1}$, with $\delta = r_t + \gamma \cdot V\left(s_{t+1}\right) \cdot m_t - V\left(s_t\right)$ and m_t a mask value of 1 when a transition completed an episode. Finally, $L_t^{VF}(\theta) = MSE\left(V_\theta\left(s_t\right), V_t^{\text{targ}}\right)$. To balance exploration and exploitation, the output of the policy in a given state is used as a multinomial probability distribution and the action is sampled with it.

The system is separated into two parts:

- **Infrastructure Part and Data source:** it comprises the industrial plant, the sensors, the PLCs and the OPC-UA server, located at Bharat Forge Kilsta;
- **Data streaming and processing:** includes a message broker that acts as a source for the EDMA and the EDMA itself, which can be placed on an edge compute platform.

For the Infrastructure and Data source part, the PLCs that control the furnace export sensor telemetry data to the local OPC-UA server, which can also manipulate the PLC by writing commands into it. By connecting our data streaming and processing functionality to the OPC-UA server, we thus implicitly can control the PLC through writing to specific tags in the OPC-UA server. In the following subsection, we summarize how the detailed components of the system were implemented, focussing on the Data streaming and processing parts.

4.3. Storage services

Considering the choice of storage service, it is crucial to carefully assess the specific requirements and determine the most appropriate technology to meet those needs. As mentioned previously, the following storages are demanded by the microservices:

- **Cache storage** to store small pieces of data from raw telemetry and the current material in production;
- **Telemetry storage** to store all telemetry records, the snapshots created of the state, and the production changes that occurred;

- **Algorithm storage** to store algorithms (binaries, scripts, or plain text files to be processed) and additional data that could be helpful for later use;
- **Power control config storage** to store which manager is active for each of the two production modes and which versions of the algorithms are loaded;
- **Forge sensors storage** to store the current production mode, the current demanded voltages in each furnace zone, and the current material in production.

To represent those storages in a systematic way, three techniques were used for storage:

- **key–value** for an easy setup, fast reading, and writing of small pieces of data. For this purpose, Redis Sutton and Barto (2018) (version 6.2.6) – which works in memory and therefore is extremely fast – is used;
- **NoSQL for structured/semi-structured data** to support storing both structured and unstructured data, focusing on writing speed and the possibility of horizontal scaling in case of need. Apache Cassandra Cagle et al. (2020) (version 4.0.3) was chosen because of its robustness and ability to scale without trouble;
- **file system** for storing files, binaries, and scripts. Nothing special was used in this case, just the plain file system where the software runs.

An instance of Redis named forge_sensors_storage is therefore reachable by all the microservices in the architecture and comprises the cache storage, power control configuration storage, and Forge sensors storage that have just been presented. Instead, the Cassandra instance maps only to Telemetry storage and is named after it: telemetry_storage. Finally, the Algorithm storage uses the file system.

4.4. Message broker

The characteristics of a message broker to allow communication among microservices to allow EDA to function well are high throughput, low latency, and the possibility of having multiple topics. For the architecture presented, it is necessary that this broker temporarily stores the messages that are sent so that if a service crashes or reboots, the data are not lost. The broker can scale horizontally when needed and the throughput is high enough to support all telemetry data to be sent almost at the same time on the same topic (about 200 records per second). Here, the writing and reading time is low to allow almost real-time data processing (in particular, it has to be possible to process all telemetry sent during the same second to try creating a state snapshot). The broker also maintains the order of the messages sent, at least under certain conditions.

All these characteristics are respected by the chosen message broker, Apache Kafka[2] (version 5.5.3 released and maintained by Confluent Inc.), which also has the good aspect of being very efficient and reliable. Since all microservices are written in Python to have a coherent technology stack, the library used to connect to Kafka is called Faust,[3] which is also one of the few libraries that supports the Kafka Streams API in Python.

4.5. Integration with the plant (OPC-UA)

Updates to OPC-UA tags will be pushed into the message broker's (i.e., Kafka) topic telemetry. The creation of this stream of data is allowed by combining different tools:

- the connection to OPC-UA is performed through a ThingsBoard IoT Gateway[4] that offers a straightforward setup and an OPC-UA connector and data visualization out-of-the-box;
- the data collected through the OPC-UA connector is converted to a preliminary JSON object and pushed to an MQTT broker (Eclipse Mosquitto[5]) to transform it into a stream of data in the publisher/subscriber fashion;
- a Kafka connector for MQTT is subscribed to the MQTT topic and eventually pushes the JSON objects as messages in a Kafka topic named v1_gateway_telemetry.

Fig. 8 shows how these components and tools communicate to integrate OPC-UA with the Kafka instance.

4.6. DRL agent and algorithm structure

The algorithm for training follows the structure of a classic RL/DRL problem, with differences in how the inner logic of the DRL agent is designed, which clearly depends on whether it is using PPO or DQN. The essential parts of it are presented through pseudocode in Algorithm 1. The syntax of the environment usage resembles that of the Python library OpenAI Gym[6] requires.

Algorithm 1: DRL-Train(env, $n_actions$, $state_size$, $episodes$, $learning_rate$, $batch_size$, $config$)

```
agent ← Agent(n_actions, state_size, learning_rate, batch_size, config)
for i ∈ 1...episodes do
  score ← 0
  done ← false
  state ← env.reset()
  while not done do
    action ← agent.choose_action(state)
    new_state, reward, done ← env.step(action)
    score ← score + reward
    agent.store_transition(state, new_state, action, reward, done)
    agent.learn()
  end while
end for
```

Fig. 9 shows a graphical representation of the Deep Q-network and Target network. The input layer has four $T_{(Z_{(3,1)})}, \ldots, T_{(Z_{(3,4)})}$ or 15 neurons $T_{(Z_{(3,1)})}, \ldots, T_{(Z_{(3,15)})}$, depending on whether the simulator is configured to use the forge or virtual sensors, for the temperatures registered in the third zone, and 1 for its power (P_{Z3}). Two fully connected, hidden layers have m and n neurons, followed by a ReLU activation function. The output layer is another fully connected layer and has three output neurons (a_1, a_2, a_3), each of them representing one action the agent can take (the output of the model is the raw value of the neuron without activation functions such as softmax).

Given A = (a_1, a_2, a_3) the output of the neural network, the action a^* the agent would take is:

$$\begin{pmatrix} a^* = argmax(a_i) - 1 \\ i \in 1, 2, 3 \end{pmatrix} \tag{6}$$

When training, a^* is chosen only with probability $1 - \varepsilon$, as defined by the $\varepsilon - greedy$ strategy.

The DT is designed to replicate the functionality of the actual furnace while simultaneously offering insights beyond those available from sensor data in the physical environment. Fig. 10 graphically represents the furnace DR.

From Fig. 10, several aspects can be discerned. The DT is initialized by loading a configuration. This includes setting parameters such as the duration of a simulation step (in seconds), the total number of simulation steps, the initial velocity of the rod (in meters per second), the starting power of the induction coils (in kilowatts), the placement of sensors and the specific start and end points of each coil (in meters,

[2] Apache Kafka. URL: https://kafka.apache.org/.
[3] Faust. URL: https://faust.readthedocs.io/en/latest/.
[4] ThingsBoard IoT Gateway. URL: https://thingsboard.io/docs/iotgateway/what-is-iot-gateway/.
[5] Eclipse Mosquitto. URL: https://mosquitto.org/.
[6] OpenAI Gym. url: https://www.gymlibrary.dev/.

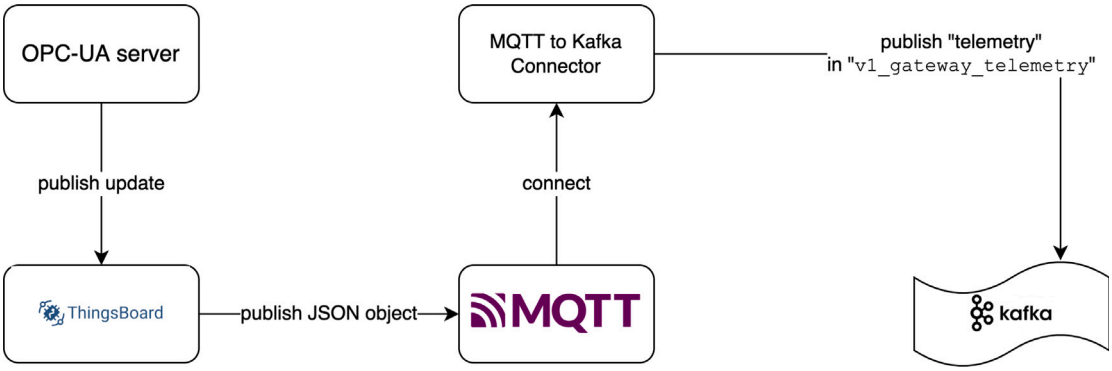

Fig. 8. How tools communicate for integrating OPC-UA with the Kafka instance.

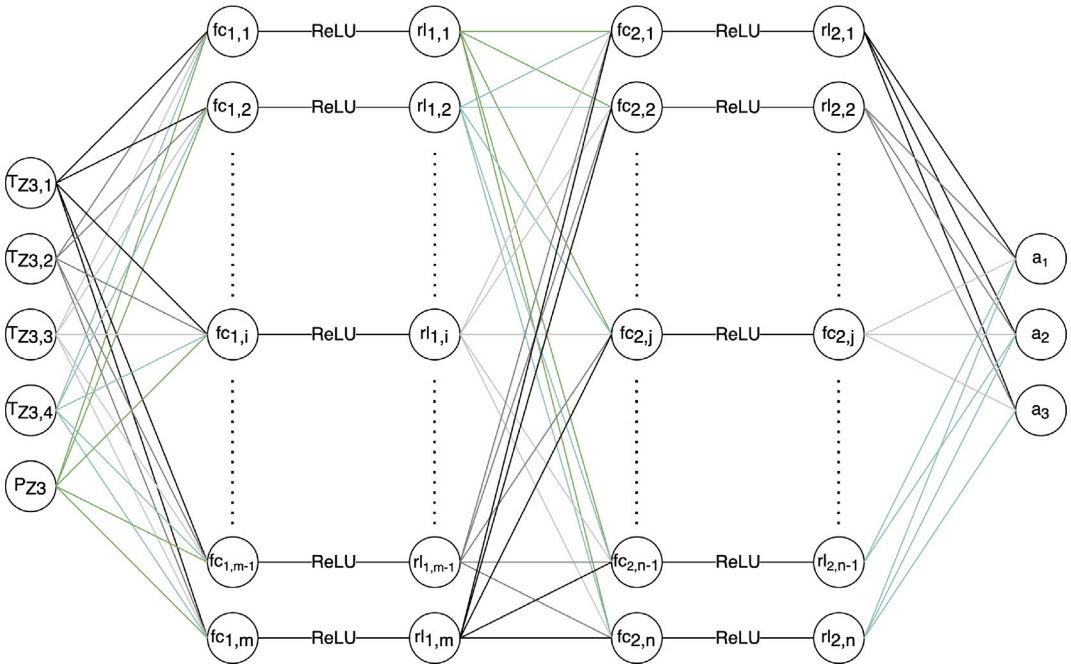

Fig. 9. The DQN graphically represented (assuming the forge sensors were used).

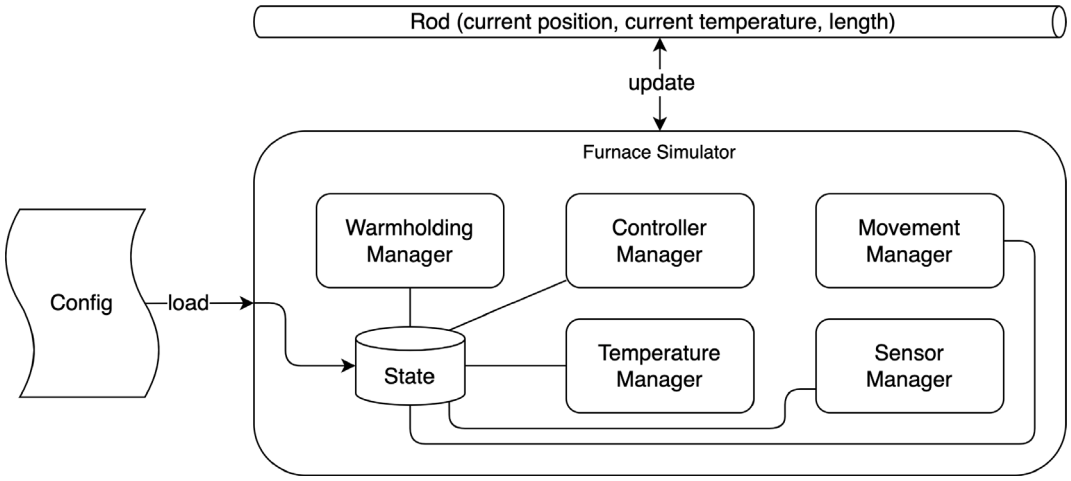

Fig. 10. A graphical representation of the furnace DT.

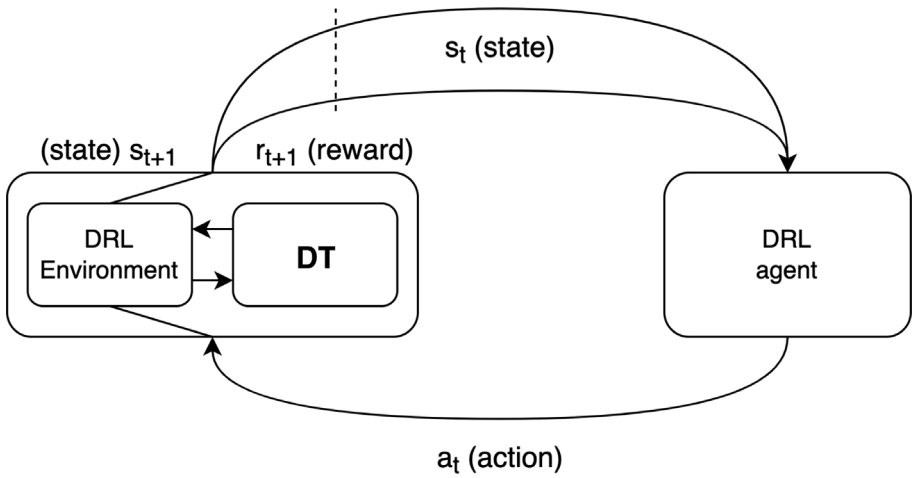

Fig. 11. The DRL cycle for the problem and its integration with the DT.

relative to the warehouse's starting point), and the ambient room temperature (in degrees Celsius). Once configured, these settings are integrated into the DT's state. The Managers are linked to the DT state, which is then updated throughout the simulation. The rod, along with the configuration, serves as a real-time input to the DT. Its position and temperature are updated at each step in real-time. Here, the Warmholding Manager monitors the status of warmholding and acts accordingly. The Controller Manager adjusts the power of the induction coils. The Movement Manager updates the position of the rod, and the Temperature Manager modifies the rod's temperature. The Sensor Manager records the temperature at each sensor location.

The DT is capable of operating over single or multiple steps. The distinction between these two modes is significant: operating step-by-step allows for control over the process at each individual time step, whereas running it over multiple steps provides control only at the conclusion of the entire process. This distinction is particularly relevant for the DRL agent, which requires the ability to manage events within the DRL environment at its own pace. Consequently, the DT is designed to accommodate various implementations of managers. In alignment with this requirement, the DRL agent typically performs the simulation for a single step, during which it evaluates the rod's position and temperature profile. Once this analysis is completed, the agent starts the next step of the simulation, using the output of the rod from the previous step as its starting point. As mentioned previously, the DRL is built on top of the DT. The DT is fed with real-time production data. Fig. 11 shows the DRL cycle for the problem and its integration with the DT.

The detailed implementation of the DT, its internal structure, and its evaluation are beyond the scope of this paper. More details about the design and implementation of the DT can be found in our earlier publication (Ma et al., 2022). DR was extensively evaluated and tested in our earlier study, which can be found in Ma et al. (2023).

5. Deployment

The deployment of our DT solution, with its DRL algorithms running within the EDMA framework, presents unique challenges and requirements. In this section, we focus on the MLOps aspects of deploying, maintaining, and updating DRL models within our DT environment, ensuring that the virtual representation remains synchronized with the physical furnace.

Model maintenance and model management updates are crucial in MLOps. With the availability of many models, only one Power Control Manager loads them. To this end, the exported models must have a shared interface to keep a level of model abstraction and allow the manager to interface with so many models' inner details. When one model is ready to be deployed, it must be wrapped in advance with layers that translate the common input to the input of the model and the output to the common output. The interface is unified between the DRL models. Its design tries to take into account all possible training needs. Since the plant has 18 temperature sensors and five power values for the five zones, the model has 23 input features. These 23 features include 18 temperature values (2 for zone 1 and 4 for zones 2 to 5) in °C, followed by five power values in kW. The interface will also take the recorded temperatures and expected powers from the state snapshots. It also takes 20 output features that are five (logistic) probability distributions representing, for each zone in ascending order, the probability of increasing, decreasing, not changing, or dropping to a very low value the power applied to each zone.

The inner logic of the exported model will also follow a unified mechanism during deployment. It removes any input feature that is not necessary and normalizes the input features in [0, 1] when needed. It also finds a function f that interpolates the temperatures of the forge sensors at the positions of the forge sensors to compute the temperatures the virtual sensors would register. Then, it builds the output for the required zones and sets to zero the remaining unused values.

For any input x, the original model m and the exported model $\acute{m}$ should always return the same value, that is, $m(x) = \acute{m}(x)$. In the case of a DRL agent m controlling zone 3 that normalizes the input and uses virtual sensors, exporting it means creating a wrapper $\acute{m}$. The wrapper $\acute{m}$ keeps the input features from the 6th to the 10th and the 21st (temperatures and power in zone 3) and finds an interpolation function f $s.t.$ $\forall i \in 1, \ldots, 4 f(fp_i) = ft_i$, considering $ft_i \forall i \in 1, \ldots, 4$ as the four temperature features and fp the positions of the corresponding forge sensors. The wrapper also computes the temperatures $vt_i = f(vo_i) \forall i \in 1, \ldots, 15$ that would be recorded by the virtual sensors at each of their positions vp_i. It is also necessary for the model wrapper to normalize vt and the value of the power feature p in the range [0, 1], resulting in $\acute{vt}$ and $\acute{p}$, and also calculate $a = m(\acute{vt} \parallel \acute{p})$ with $a \in \mathbb{R}^3$ corresponding to three possible actions. Fig. 12 shows a graphical representation of the wrapper behavior.

6. Experimental evaluation

In this section, we evaluate our integrated system that combines the DT, Event-Driven Microservices Architecture, and DRL algorithms. While our evaluation focuses primarily on the performance of the DRL algorithms, it is important to note that these results are achieved within the context of our DT environment, which is enabled by the EDMA infrastructure described earlier.

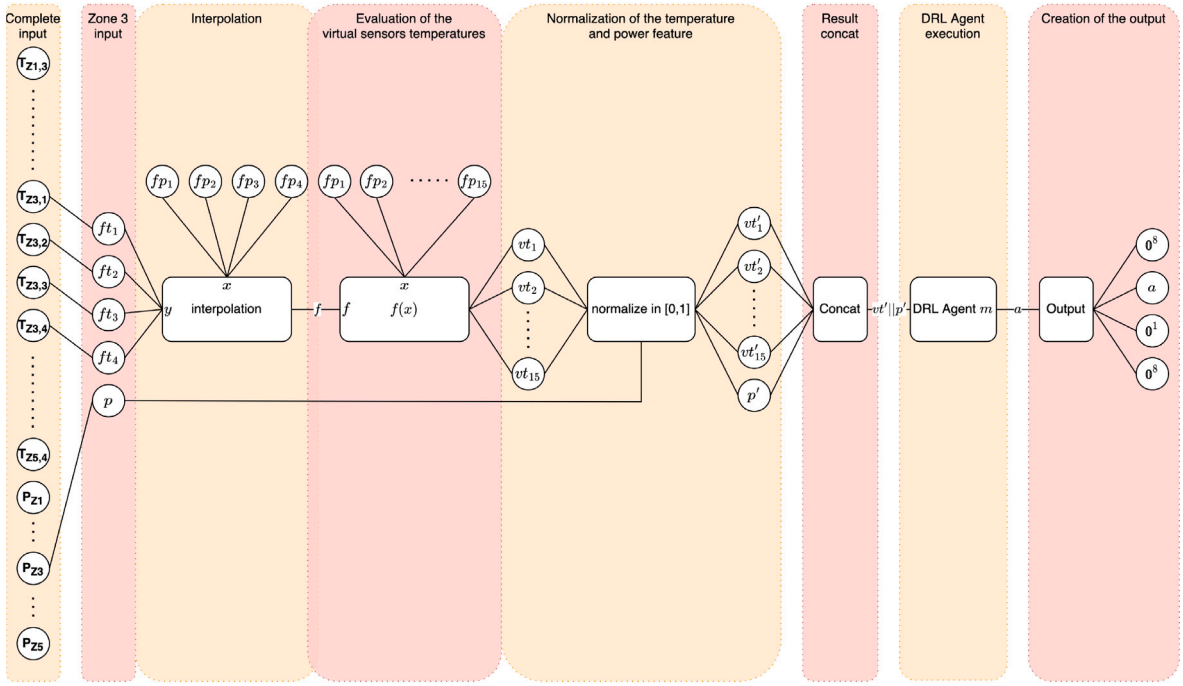

Fig. 12. Graphical representation of an example of a model wrapper and deployment.

Here, we aim to conduct an empirical assessment to evaluate the operational performance and compliance of the Event-Driven Microservices Architecture (EDMA). The primary objectives of this assessment are to determine the extent to which the EDMA aligns with its intended functionality, ensure that the microservices implementations adhere to the specified design requirements, and verify that they operate within expected timeframes.

The experimental data presented in this section were gathered using a computing environment with the following hardware and software specifications:

- Operating system: Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-124-generic x86_64)
- CPU: Intel Xeon Silver 4110 CPU @ 2.10 GHz (8 cores)
- RAM: 46 GB
- Docker: 20.10.12, build e91ed57

Our investigation aims to assess whether the EDMA functions as intended by examining the microservices' correct behavior and performance within this specific computational environment.

6.1. Processing time: Maximum expected vs. Actual average

To ensure the correctness and effectiveness of microservices, it is imperative to assess their actual performance against their expected processing times. This evaluation focuses on explaining the specified time windows within which each microservice is intended to run. For the three main microservices, it is important that the microservices comprising the Telemetry Parser effectively process incoming sensor data, which may reach up to 200 records per second. Consequently, it is crucial that each sensor's data, including the processing required for contributing to state snapshots, be handled within a maximum time frame of 5 ms. For the Power Control module, particularly the DRL Normal Production Manager, it is necessary to process approximately one snapshot per second. Within the Data Manager component, the Power Updater must process approximately one power update per second. This ensures that the plant's operational parameters are promptly updated on the basis of recent snapshots. In addition, it is imperative to maintain up-to-date information on the current voltages applied to the induction coils and to update the active production mode in response to any changes.

Table 1
Comparison of maximum expected and actual processing times.

Microservice	Max proc. time (ms)	Actual proc. time (ms)
Telemetry parser	5	4.622
Power control	1000	4.055
Data manager	1000	9.922

The processing time thresholds for each microservice were determined based on the operational requirements of the steel production process and the technical constraints of our system:

1. **Telemetry Parser (5 ms threshold):** This component must rapidly parse incoming sensor telemetry and organize it for further processing. The threshold ensures smooth data flow without backlog buildup.
2. **Power Control (1000 ms threshold):** The induction furnace's thermal dynamics dictate this threshold. Our process engineers at Bharat Forge Kilsta determined that updating power settings more frequently than once per second would not significantly improve control but could potentially cause system instability.
3. **Data Manager (1000 ms threshold):** This aligns with the Power Control threshold to ensure that each power update is promptly communicated to the OPC-UA server, maintaining synchronization between our control system and the physical furnace.

These thresholds represent the maximum allowable processing times to maintain real-time control and system stability in the steel production environment.

The actual processing times have been derived from an extensive collection of data spanning two to three months, obtained through comprehensive log analysis, ensuring the reliability and accuracy of the presented results. These empirical findings are detailed in Table 1.

As can be seen in the table, the processing time for the Telemetry Parser begins when events are published on Kafka. Since this microservice does not interact directly with external systems like OPC-UA, its processing time depends mainly on internal operations and

Kafka message handling. It may not fully capture the network-related latency between the OPC-UA server and the EDMA. The Power Control microservice performs remarkably efficiently, even considering the network-related latency. Despite potential delays in data retrieval or communication with external systems, it still manages to process snapshots well within the expected time frame. This suggests that it effectively handles any inherent latency challenges. The higher average processing time for the Data Manager, especially the Power Updater microservice, can be attributed to its need to connect to OPC-UA to send power updates. The network latency introduced by the VPN connection likely contributes significantly to this increased processing time. Despite having less computational work to perform, the impact of network delays becomes more pronounced in this microservice.

6.2. DRL power control algorithm

In this section, we present the results obtained from a series of training jobs conducted for the power control algorithm within Zone 3 of the furnace. We chose to focus on Zone 3 for several key reasons. Firstly, Zone 3 operates in a critical temperature range (1140 °C to 1275 °C, as shown in Fig. 2) where precise control is crucial for product quality. Secondly, this zone represents a transition point between the intensive heating of the first two zones and the final temperature stabilization in the last two zones, making it particularly challenging to control. Moreover, the control challenges in Zone 3 are representative of the overall furnace control problem, providing insights that can be generalized to other zones. Additionally, given the computational intensity of DRL training, focusing on one zone allowed for a more thorough exploration of hyperparameters and algorithms within our available resources. Although we conducted experiments for all zones, the results from Zone 3 provide a comprehensive view of our DRL approach's effectiveness. The insights gained from this zone can be applied to optimize control strategies across the entire furnace.

These experiments included a wide range of hyperparameter configurations, variations in the loading of the DRL environment, and the reward function used. The primary objectives of this section are to determine the optimal algorithm for various scenarios, quantify the reduction in execution time achieved by using the simulator environment, and identify reward functions that produce improved temperature profiles. The best model will then be deployed for inference.

The experiments were conducted on the "Berzelius" compute cluster, administered by the National Supercomputer Center (NSC) at Linköping University, Sweden. The node specifications for this cluster are as follows:

- Operating system: RHEL 8 (4.18.0-348.20.1.el8_5.x86_64)
- CPU: AMD EPYC 7742 64-Core Processor (16/64 cores assigned)
- RAM: 125/1000 GB assigned
- GPU: NVIDIA A100-SXM4-40 GB
- CUDA drivers: 11.2
- Python: 3.10.4
- PyTorch: 1.12.0+cu113

Subsequent analysis in the following subsections will dive into the findings, highlighting the performance differences between algorithms, the time reductions afforded by the DT, and the impact of reward functions on achieving optimal temperature profiles in Zone 3.

6.2.1. Collected metrics

In each training job, we systematically collect various metrics about both the performance of the algorithm, including the "status" of the DNN, and the outcomes generated within the DRL environment. These metrics encompass the following data points for each experiment:

- **Selected Hyperparameters:** The configuration parameters chosen for the experiment are fundamental to influence the behavior and learning of the algorithm.
- **Episode Score:** This metric represents the cumulative rewards acquired during each episode. It provides information on how well the agent achieves its objectives.
- **Mean Loss per Episode:** The mean loss value computed for each episode. This calculation aggregates losses incurred during the episode, reflecting the algorithm's convergence or divergence behavior.
- **Mean Step Time per Episode:** The average time taken per step within an episode. It helps gauge the temporal efficiency of the algorithm's decision-making process.
- **Total Episode Time:** The cumulative duration of each episode indicates how long the agent took to complete its decision-making process within the environment.
- **Number of Steps per Episode:** The count of discrete steps taken within each episode. This number is typically fixed according to the current rod configurations and offers insights into the depth of exploration the agent engages in.
- **Temperature at the Last Sensor in Zone 3 (with every 500 steps):** The temperature recorded by the final virtual sensor in Zone 3, sampled at regular intervals of 500 steps. These data provide an ongoing snapshot of the temperature dynamics within the controlled zone.
- **Power Applied to Induction Coils in Zone 3 (Every 500 steps):** The magnitude of the power applied to the induction coils in Zone 3, also measured at regular intervals of 500 steps. This information is crucial to understanding the control actions executed by the agent.
- **Epsilon Value (DQN only):** For the $\epsilon-greedy$ strategy used by the DQN algorithm, we monitor the current value of ϵ, which influences the exploration-exploitation trade-off. This value guides the agent's propensity to take random actions versus exploiting known policies.

Collecting and analyzing these metrics across various experiments allows a comprehensive assessment of algorithmic performance, hyperparameter sensitivity, convergence behavior, and the impact of chosen configurations on the DNN and DRL environment.

6.2.2. Hyperparameters and episodic score

Each training job is initiated with a unique combination of hyperparameters primarily dependent on the choice of algorithm (DQN or PPO). However, certain hyperparameters are shared across all jobs to ensure consistency in the evaluation process. During the experiments, we cover the "Number of Episodes" to determine the number of training episodes for each job. To specify the learning rate of the optimizer used during training, we use the "Learning Rate" parameter. We also use the "Random Seed" parameter for libraries that generate pseudo-random numbers to ensure reproducibility. To dictate the number of transitions selected from memory at each training step, we use the "Batch Size" parameter. We use "Normalize Observations", which is a Boolean flag that indicates whether observations should be normalized during training. We also used a Boolean flag that determines whether noise should be applied to the power levels of the first two zones and a Boolean flag that allows for the choice between virtual or forge sensors for data acquisition.

Table 2 presents the permissible values that each of these common hyperparameters can assume during a training job, providing information on the flexibility and configurability of the training process.

For the DQN algorithm, a distinct set of hyperparameters is customized to optimize its performance. These hyperparameters are essential for shaping the behavior and learning process of the DQN agent. We used γ **(Gamma)** that determines the discount factor for the Bellman equation, which influences the agent's preference for immediate rewards versus long-term rewards. **Starting** ϵ **(Epsilon)** is used to signify the initial value of the ϵ-greedy exploration strategy. It regulates the agent's propensity to explore the environment versus

Table 2
Common hyperparameter values.

Hyperparameter	Values
Episodes	50, 100, 200, 300
Learning rate	0.001, 0.0001, 0.00001
Seed	19, 39
Batch size	64, 128, 256, 512, 1024, 2048, 4096
Normalization	Enabled (True), Disabled (False)
Z1 and Z2 powers	With noise (False), Without noise (True)
Sensors	Virtual (False), Forge (True)

Table 3
DQN hyperparameter values.

Hyperparameter	Values
γ	0.9, 0.95, 0.98, 0.99
ϵ in ϵ-greedy	0.7, 0.8, 0.9, 1.0
ϵ's lower bound	0.01, 0.001, 0.0001, 0.00001
ϵ's decrease step	0.05, 0.005, 0.0005, 0.00005
Fully connected 1 neurons	128, 256, 512
Fully connected 2 neurons	128, 256, 512
Target network update interval	1000, 10 000, 100 000
Transitions memory	100 000, 200 000, 500 000

exploiting its current knowledge. **Lower Bound for** ϵ is also used to set the minimum value that ϵ can reach during the training process. The **Decrease Step for** ϵ is used to specify the rate at which ϵ is reduced over time. **DNN's First Fully Connected Layer Neurons** is used to determine the number of neurons in the first fully connected layer of the Deep Neural Network (DNN) architecture used to estimate Q values. Similarly, **DNN's Second Fully Connected Layer Neurons** is used to determine the number of neurons in the second fully connected layer of the DNN. The DQN algorithm employs a target network to stabilize the training. The **Number of Steps Before Updating Target Network Weights** θ' defines the frequency at which the weights of the target network θ' are updated. Finally, the **Transitions Memory Size (Replay Memory Size)** parameter is used to define the size of transition memory (often referred to as replay memory), which stores experiences for replay during training.

These DQN-specific hyperparameters are critical in configuring the DQN agent's learning dynamics and overall training performance, allowing for fine-tuned adjustments to address specific task requirements and challenges. Table 3 shows the values each DQN hyperparameter can assume during a job.

In the context of the PPO algorithm, several hyperparameters come into play to fine-tune and customize the training process. The λ (Lambda) Value represents the weight applied to the Generalized Advantage Estimate (GAE). The coefficient c_1 plays an important role in shaping the PPO loss function. The ϵ (epsilon) determines the extent to which policy updates are constrained, as detailed in the aforementioned section. The number of training epochs executed during each training round that influences the convergence and stability of the learning process. The Number of Steps Before Training defines the number of steps taken before retraining the Actor and Critic networks and the size of the memory of the transition. The Actor's First Fully Connected Layer Neurons parameter is used to specify the number of neurons in the first fully connected layer of the Actor-network affects the network's capacity to represent policy functions. Similarly to the first layer, the number of neurons in the second fully connected layer of the actor network influences the policy function's expressive power. Table 4 shows the values that each PPO hyperparameter can assume during a job.

The results presented have been obtained through the implementation of DRL agents for the two aforementioned scenarios of normal production. We conducted a comprehensive hyperparameter search, systematically exploring the combinations of values detailed in Tables 2, 3, and 4. This exploration included testing 4 different episode counts, 3 learning rates, 2 random seeds, and 7 batch sizes for common

Table 4
Values PPO hyperparameters.

Hyperparameter	Values
λ for GAE	0.9, 0.95, 0.98, 0.99, 1.00
c_1 in loss	0.5, 1.0
ϵ in the policy clip	0.1, 0.2, 0.3
Epochs	5, 10, 15, 20, 25, 30
Training interval	10, 25, 50, 100, 200, 500, 1000, 2000, 5000
Actor fully connected 1 neurons	128, 256, 512
Actor fully connected to 2 neurons	128, 256, 512
Critic fully connected 1 neurons	128, 256, 512
Critic fully connected 2 neurons	128, 256, 512

parameters (Table 2). For DQN, we explored 4 values for γ, 4 for initial ϵ, and various other algorithm-specific parameters (Table 3). Similarly for PPO, we tested 5 values for λ, 3 for ϵ in the policy clip, and multiple configurations for network architectures (Table 4). This systematic approach resulted in a large number of distinct experiments that covered a wide range of configurations for each algorithm (DQN and PPO) and reward function (symmetric, asymmetric, and hyperbolic). For each scenario, we performed two analyses: one on the hyperparameters and another revealing the best models achieved. Our initial analysis focused on identifying the most crucial hyperparameters and determining which configurations for DQN and PPO yield optimal outcomes. In addition, our goal was to gain insight into the performance of each reward function by individually examining them.

We utilize correlation matrices, a powerful mathematical tool, to analyze the extent to which hyperparameters are correlated with the best score. We only consider correlations with an absolute value greater than 0.15. Any results that appear promising or align with the correlation are then selected.

In the following tables, we present the scores achieved by our DRL models during testing. These scores represent the cumulative rewards obtained by the agent over an episode, indicating how well the model performs in maintaining the desired temperature profile. Higher scores indicate better performance. The Symmetric and Asymmetric reward functions typically yield scores in the hundreds, while the Hyperbolic reward function produces scores between 0 and 1 due to its different scaling. It is important to note that these scores are directly comparable only within the same reward function type.

In a typical production scenario, certain correlations have been observed that hold a significant importance. These correlations are measured and exhibit a noticeable connection between different parameters and the resulting performance. Table 5 summarizes the results of these findings.

These correlations are derived from the analysis of the PPO DQN algorithms within the specified production scenario. The coefficients provide insights into how changes in these parameters could affect the overall performance of the algorithms. Positive coefficients indicate a direct relationship, where an increase in the parameter correlates with an increase in performance. In contrast, negative coefficients suggest an inverse relationship, implying that a decrease in the parameter value could potentially lead to improved performance. The data specifically highlight the nuanced effects of the neural network architecture (such as the number of neurons in fully connected layers) and algorithmic hyperparameters (such as ϵ in the policy clip and c1 in loss) on the efficacy of the machine learning models.

Table 6 displays the outcomes achieved using various hyperparameters, namely (PPO) Epochs, (PPO) Training interval, Use forge sensors, and ϵ in ϵ-greedy. This table presents the average scores the PPO algorithm achieves for different numbers of training epochs in the normal production scenario. The scores are shown for three different reward functions: Symmetric, Asymmetric, and Hyperbolic. Higher scores indicate better performance, with the Symmetric and Asymmetric functions typically yielding scores in the hundreds, while the Hyperbolic function produces scores between 0 and 1. The variation

Table 5
Correlation coefficients in normal production scenario.

Parameter	Correlation coefficient	Interpretation
PPO epochs	0.4230	Positive correlation
PPO training Interval	0.3923	Positive correlation
PPO actor fully connected 1 neurons	−0.3262	Inverse correlation (Lower value, higher score)
PPO ε in policy clip	0.3161	Positive correlation
PPO c1 in loss	0.2653	Positive correlation
Use forge sensors	−0.1936	Inverse correlation (Using forge sensors, lower score)
DQN transitions memory	−0.1904	Inverse correlation
PPO critic fully connected 1 neurons	−0.1771	Inverse correlation
DQN ε in ε-greedy	−0.1529	Inverse correlation

Table 6
PPO Epochs Analysis in normal production: Test scores achieved with different reward functions. Scores represent cumulative rewards over an episode, with higher values indicating better performance in maintaining the desired temperature profile.

(PPO) epochs	Symmetric	Asymmetric	Hyperbolic
5	439.711	329.045	0.129
10	529.712	477.993	0.216
15	472.257	111.429	0.324
20	559.902	684.969	0.522
25	446.595	689.504	0.227
30	504.743	644.979	0.643

Table 7
PPO Training Interval Analysis: Test scores for different reward functions in normal production. Scores indicate the model's ability to maintain the desired temperature profile, with higher values representing better performance.

(PPO) training interval	Symmetric	Asymmetric	Hyperbolic
10	731.871	301.635	0.171
25	572.918	287.197	0.486
50	439.64	268.613	0.366
100	485.41	542.401	0.123
200	379.149	665.818	0.525
500	524.616	130.409	0.193
1000	398.955	687.44	0.427
2000	498.443	684.969	0.313
5000	606.455	691.889	0.151

in scores across epochs demonstrates the impact of the number of training iterations on model performance. The results indicate that the score increases with an increasing number of training epochs. This could be attributed to the fact that the Actor and Critic DNNs can reduce the agent's loss more efficiently. Additionally, it is worth noting that the average result obtained by the jobs with the hyperbolic reward function is considerably high.

Table 7 shows the average scores achieved by the PPO algorithm for different training intervals in the normal production scenario. The training interval represents the number of steps taken before retraining the model. Scores are presented for three reward functions: Symmetric, Asymmetric, and Hyperbolic. Higher scores indicate better performance, with Symmetric and Asymmetric functions typically yielding scores in the hundreds, while the Hyperbolic function produces scores between 0 and 1. The variation in scores across training intervals illustrates how the frequency of model updates affects performance for each reward function. The results are presented with respect to the training interval of the agent, specifically, the frequency at which the agent retrains DNN. Within the PPO, this training interval is intrinsically linked to the number of transitions accumulated in the PPO's transition memory. A discernible trend emerges from the data: as the number of steps (and consequently, the number of transitions) increases, there is a corresponding rise in the average score achieved by the agent.

For the scenario after warmholding, the hyperparameters with higher correlation coefficients are as in Table 8. Finally, the results obtained with the following hyperparameters: (PPO) Actor F.C. 1 neurons, No noise before zone 3, (PPO) c1 in loss. The others are not presented, as they do not significantly contribute to the performance.

Table 8
Hyperparameters and settings for after warmholding scenario.

Parameter	Value
(DQN) γ	−0.3845
(PPO) Actor fully connected 1 neurons	0.3545
(PPO) Actor fully connected 2 neurons	0.3292
No noise before zone 3	−0.3153
(PPO) c_1 in loss	0.1786

Table 9
PPO Actor Network Analysis: Test scores for different reward functions in normal production after warmholding. Scores reflect the model's performance in maintaining the desired temperature profile after transitioning from warmholding, with higher values indicating better performance.

(PPO) actor F.C. 1 neurons	Symmetric	Asymmetric	Hyperbolic
128	97.422	161.654	0.165
256	183.306	172.646	0.254
512	196.603	181.276	0.098

Table 10
Best models obtained in the proper normal production scenario.

Algo.	Rew. fun.	Best score	Mean score (same fun.)	Mean score (hyperb. fun.)
DQN	Sym.	796.469	704.804	0.510
DQN	Asym.	689.968	709.283	0.475
DQN	Hyperb.	0.621	0.547	0.443
PPO	Sym.	792.307	725.609	0.542
PPO	Asym.	727.665	756.205	0.526
PPO	Hyperb.	0.643	0.538	0.469

As in the case of Tables 6 and 7, Table 9 shows the average scores obtained with the various values of neurons in the first fully connected layer of the Actor DNN. The correlation coefficient matches the results, except for the average score using 512 neurons with the hyperbolic reward function.

6.2.3. Best model

We conducted a series of experiments to identify the optimal model by fine-tuning hyperparameters for each algorithm. In Table 10, we present the findings of these experiments in relation to the normal production scenario. On the basis of these results, we have determined that the model generated with PPO and the symmetric reward function is the most effective. This decision was not based on its performance during training (as DQN with a symmetric reward function achieved the best score), but rather on its consistently high mean score during testing with both the symmetric and hyperbolic functions, indicating that it closely aligns with the desired temperature profile.

Table 11 shows the hyperparameters with which the best model was trained. To know how the best model performs in normal production, Fig. 13 shows the temperature profile and the running results of the best model in normal production.

As for the best model in the normal production after warmholding scenario, Table 12 show the comparison of the score for both DQN and PPO algorithms. Although none of the models presented in this table exhibit clear superiority over the others, it is worth noting that the DQN

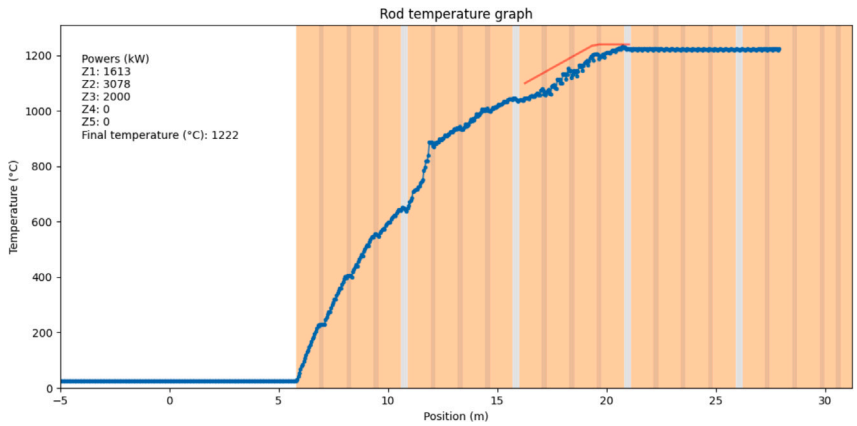

Fig. 13. Running the model in proper normal production.

Table 11
Hyperparameters of the best model in proper normal production.

Hyperparameter	Value	Hyperparameter	Value
Episodes	100	c_1 in loss	1
Learning rate	0.001	ε in policy clip	0.2
Seed	19	Epochs	20
Batch size	4096	Training interval	100
Normalization	Enabled	Actor F.C. 1 neurons	256
Z1 and Z2 powers	Without noise	Actor F.C. 2 neurons	512
Sensors	Virtual	Critic F.C. 1 neurons	256
λ for GAE	1	Critic F.C. 2 neurons	256

Table 12
Best models obtained in the normal production after warmholding scenario.

Algo.	Rew. fun.	Best score	Mean score (same fun.)	Mean score (hyperb. fun.)
DQN	Sym.	229.112	227.932	0.281
DQN	Asym.	208.230	170.357	0.157
DQN	*Hyperb.*	0.307	0.274	0.298
PPO	Sym.	228.791	231.479	0.293
PPO	Asym.	209.637	204.991	0.279
PPO	*Hyperb.*	0.301	0.301	0.270

Table 13
Hyperparameters of the best model in normal production after warmholding.

Hyperparameter	Value	Hyperparameter	Value
Episodes	300	F.C. 2 neurons	256
Learning rate	0.00001	Target net. update interval	100 000
Seed	19	Transitions memory	100 000
Batch size	1024	ϵ in ϵ-greedy	0.7
Normalization	Disabled	ϵ's lower bound	0.01
Z1 and Z2 powers	With noise	ϵ's decrease step	0.05
Sensors	Virtual	γ	0.9
F.C. 1 neurons	512	–	–

model with hyperbolic function performed well and shows promising potential. Therefore, it has been selected as the optimal model. For reference, the hyperparameters used during its training are listed in Table 13.

The empirical data presented in Fig. 14 provides a snapshot of the model's performance during testing. The model demonstrated a higher degree of ability to fit the desired temperature profile.

7. Impact, generalizability, and lessons learned

Our integrated approach—combining DT modeling, EDMA for real-time processing, and DRL for intelligent control, has significant potential to improve steel production processes. While each component contributes unique value, it is their combined implementation that enables the improvements discussed below.

Our DT-based approach, which uses DRL, has significantly improved steel production processes' control. Although direct measurements of sustainability, efficiency, and cost-effectiveness are not provided, we can infer these improvements from our experimental results. The measurement of waste reduction in industrial settings like steel production faces several challenges that prevented direct quantification in this study. These include production priorities that often limit opportunities for experimental control groups, operational constraints that restrict prolonged data collection periods, resource limitations affecting sensor deployment, and the complex nature of waste, which can manifest as energy inefficiency, material scrap, or quality defects. In addition, proprietary concerns in industrial partnerships often limit the disclosure of certain performance metrics. Despite these constraints, our experimental results provide strong indirect evidence of improvements through the temperature profile optimizations shown in our testing.

7.1. Efficiency and quality improvements

The DRL-powered control system has shown substantial improvements in maintaining optimal temperature profiles during steel production. In the normal production scenario, our best model achieved a high degree of alignment with the desired temperature profile, as evidenced in Fig. 13. This precise temperature control likely leads to improved product quality and reduced energy waste from overheating. Furthermore, in the challenging scenario of normal production after warmholding, Fig. 14 demonstrates our model's ability to effectively manage temperature profiles even when starting from non-uniform temperature distributions. This adaptability suggests improved efficiency in managing production variations and potentially reduced waste from suboptimal heating patterns.

7.2. Sustainability and cost-effectiveness implications

The precise temperature control demonstrated in both scenarios indicates more efficient energy use in heating. Our system reduces unnecessary energy consumption and associated costs by maintaining optimal temperatures without overheating. The improved control, particularly in challenging scenarios like post-warmholding, suggests a potential reduction in material waste due to temperature-related quality issues. Furthermore, our DRL approach demonstrated faster convergence to optimal temperature profiles, which reduces overall processing time and associated energy costs. The precise control also suggests a potential increase in the percentage of products that meet quality standards, thereby improving the yield and reducing the costs associated with rework or scrap.

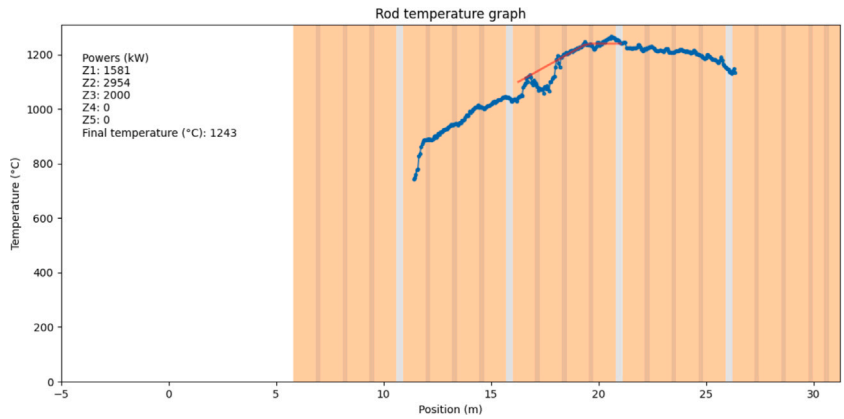

Fig. 14. Running the model in normal production after warmholding.

7.3. Use case scenario: Warmholding optimization at Bharat Forge Kilsta

We implemented our DRL-powered digital twin approach at Bharat Forge Kilsta AB to address the challenge of uneven temperature distributions in steel rods after warmholding, which may lead to a "zebra pattern" problem if not addressed properly. Using data from their furnace sensors, we initialized our model. We demonstrated a significant improvement in the management of temperature profiles in the digital twin during the transition from warmholding to normal production, as shown in Fig. 14. This real-world application potentially reduces time, energy consumption, and material waste in their production process.

Due to the sensitive nature of industrial processes, we cannot share the full implementation details or source code. However, researchers can reproduce a similar approach by initializing a DT with warmholding temperature patterns (Section 4.6), applying the DRL model framework (Sections Section 4). While exact replication may not be possible, these guidelines provide a foundation for adapting our methodology to similar steel production scenarios, balancing reproducibility with the necessary industrial confidentiality.

7.4. Generalizability and lessons learned

While our implementation is tailored to steel production at Bharat Forge Kilsta, several key lessons and generalizable approaches emerged from this work that can benefit other industries and applications:

1. Architectural Patterns for Industrial ML: The separation of concerns in our MLOps-driven architecture (data input layer, event-driven microservices, and model management) provides a blueprint that can be adapted across various industrial control systems beyond steel production. This pattern is particularly valuable in environments with strict latency and security requirements and legacy equipment integration needs.
2. DT Integration Strategy: Our approach to building a DT that supports both simulation and real-time control offers a framework that can be applied to other continuous manufacturing processes, such as chemical processing, paper manufacturing, or glass production. The design choices for simulator managers (Section 4.6) represent a generalizable pattern for building adaptable DT.
3. MLOps Workflow in Industrial Settings: The deployment and model management approach described in Section 5 provides practical guidance for managing ML models in industrial settings with limited connectivity, emphasizing model wrapping techniques that can be applied regardless of the specific industry.
4. Reward Function Design Insights: The comparative analysis of reward functions (Section 6.2) offers valuable insights for other RL applications in industrial control, showing how different reward formulations can significantly impact control quality and adaptation capabilities.
5. Edge Computing Requirements: Our work demonstrates the practical necessity of edge computing for industrial ML applications with strict latency requirements, providing a reference architecture that balances computational needs with deployment constraints common across manufacturing environments.

8. Conclusion and future work

In this paper, we have presented an innovative approach that integrates DT technology, Event-Driven Microservices Architecture, and DRL to improve steel production. Each component played a vital role: the DT provided a virtual representation of the physical process, the EDMA enabled real-time data processing and model deployment, and DRL algorithms optimized control decisions within this framework. In addition, we have presented an innovative approach that leverages the power of DRL and MLOps to improve production. Our primary focus has been automating the normal production mode, recognizing that various algorithms may be required for different production modes. Instead of going into algorithmic complications, we prioritized the architecture and infrastructure that form the backbone of our solution. The application of DRL to control the electrical power consumed during the heating process of an induction furnace has yielded good results. Our system has the potential to substantially reduce waste, improve environmental sustainability, and improve energy efficiency by continuously adapting to dynamic operational conditions in real-time. This flexibility aligns with the growing demand for sustainable and environmentally friendly manufacturing practices. Moreover, our work serves as a bridge between two distinct domains: machine learning and industrial operations. We have demonstrated that MLOps can play a key role in orchestrating the transformation of traditional industrial processes into smart, autonomous systems. This integration allows for effectively deploying and managing machine learning models in production environments.

While our research has made significant strides in reshaping the steel production landscape, it is important to recognize that there are several avenues for future work and exploration. Future research should focus on developing and integrating algorithms for different production modes beyond normal operation, such as optimizing start-up procedures, handling emergency shutdowns, and managing maintenance cycles. Additionally, exploring multi-objective optimization techniques could enable simultaneous consideration of factors like product quality, equipment lifespan, and production schedules alongside our current focus on energy efficiency and waste reduction. The potential of transfer learning in industrial settings is another promising area, which could allow the application of our DRL models to different types of furnaces or even other steel production processes with minimal retraining. Developing methods to make the decisions of our DRL models more interpretable to human operators through explainable AI techniques

could enhance trust and facilitate wider adoption in industrial settings. In terms of advancing our MLOps practices specifically, future work should focus on enhancing reproducibility through comprehensive version control systems for code, data, and models. We also aim to explore the integration of Continuous Integration and Continuous Deployment (CI/CD) practices to automate model building, testing, and deployment processes, streamlining our workflow and ensuring consistent quality in model updates. Another critical area for future investigation is the development of robust systems for monitoring and tracking the performance of our models in production settings, including real-time performance metrics, drift detection, and automated alerts for model degradation.

Further research is needed to test the scalability of our approach to the real production facility and its generalizability to other heavy industries beyond steel production. Additionally, we plan to investigate techniques for automated model retraining based on performance metrics and new data, ensuring our models remain effective over time. Future work should also include more rigorous statistical analysis of the correlations between hyperparameters and algorithm performance, potentially employing advanced statistical methods suitable for multi-dimensional hyperparameter spaces. Finally, extended studies are required to assess the long-term performance of the system and its ability to adapt to gradual changes in equipment efficiency or production requirements over time.

Heavy industry, including steel production, remains an underexplored domain in MLOps. More studies are needed that not only explore the theoretical aspects, but also present concrete results and lessons learned from the implementation of customized MLOps solutions. Our work underscores the transformative potential of MLOps and DRL in industrial settings, emphasizing the importance of focusing on infrastructure and architecture to show the way for smarter and more efficient production processes. The future directions outlined above represent exciting opportunities to build on this foundation and further advance the intelligent manufacturing field, paving the way for more sustainable, efficient, and intelligent industrial processes. To support the replicability of our work, we have provided a detailed use case scenario in Section 7.3, which offers guidelines for reproducing our approach in similar industrial settings. Although full replication may be challenging due to the sensitive nature of industrial data, we believe that this use case, combined with the detailed descriptions throughout the paper, provides a solid foundation for adapting our methods to other contexts in heavy industry.

CRediT authorship contribution statement

Bestoun S. Ahmed: Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation, Formal analysis, Conceptualization. **Tommaso Azzalin:** Writing – review & editing, Software, Methodology, Investigation, Data curation. **Andreas Kassler:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition, Formal analysis. **Andreas Thore:** Writing – review & editing, Validation, Software, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Hans Lindbäck:** Writing – review & editing, Validation, Resources, Investigation, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Andreas Kassler reports financial support was provided by KK-stiftelsen. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was partially funded by Vinnova, Sweden through the SmartForge project. Additional funding was provided by the Knowledge Foundation of Sweden (KKS) through the Synergy Project AIDA - A Holistic AI-driven Networking and Processing Framework for Industrial IoT (Rek:20200067) and by the Free State of Bavaria through the HighTech Agenda (HTA).

Data availability

The authors do not have permission to share data.

References

Branca, T.A., Fornai, B., Colla, V., Murri, M.M., Streppa, E., Schröder, A.J., 2020. Current and future aspects of the digital transformation in the European steel industry. Matériaux & Tech. 108 (5–6), 508. http://dx.doi.org/10.1051/mattech/2021010.

Cagle, M.N., Yılmaz, K., Doğru, H., 2020. Digitalization of business functions under industry 4.0. In: Digital Business Strategies in Blockchain Ecosystems: Transformational Design and Future of Global Business. Springer International Publishing, Cham, pp. 105–132. http://dx.doi.org/10.1007/978-3-030-29739-8_6.

Chai, T., Jia, Y., Wang, H., Su, C.-Y., 2017. Dual rate adaptive control for an industrial heat supply process using signal compensation approach. In: Proceedings of the 2017 American Control Conference. ACC, pp. 1234–1239. http://dx.doi.org/10.23919/ACC.2017.7963123.

Elallid, B.B., Benamar, N., Hafid, A.S., Rachidi, T., Mrani, N., 2022. A comprehensive survey on the application of deep and reinforcement learning approaches in autonomous driving. J. King Saud Univ.- Comput. Inf. Sci. 34 (9), 7366–7390.

Faubel, L., Schmid, K., Eichelberger, H., 2023. Mlops challenges in industry 4.0. SN Comput. Sci. 4 (6), http://dx.doi.org/10.1007/s42979-023-02282-2.

Ferreira Neto, W.A., Virgínio Cavalcante, C.A., Do, P., 2024. Deep reinforcement learning for maintenance optimization of a scrap-based steel production line. Reliab. Eng. Syst. Saf. 249, 110199. http://dx.doi.org/10.1016/j.ress.2024.110199.

Ferreiro, S., Konde, E., Fernández, S., Prado, A., 2016. INDUSTRY 4.0: Predictive intelligent maintenance for production equipment. PHM Soc. Eur. Conf. 3 (1), http://dx.doi.org/10.36001/phme.2016.v3i1.1667.

Graessler, I., Poehler, A., 2017. Integration of a digital twin as human representation in a scheduling procedure of a cyber-physical production system. In: 2017 IEEE International Conference on Industrial Engineering and Engineering Management. IEEM, pp. 289–293.

He, K., Wang, L., 2017. A review of energy use and energy-efficient technologies for the iron and steel industry. Renew. Sustain. Energy Rev. 70, 1022–1039.

Huang, X., Hong, S.H., Yu, M., Ding, Y., Jiang, J., 2019. Demand response management for industrial facilities: A deep reinforcement learning approach. IEEE Access 7, 82194–82205.

Kano, M., Nakagawa, Y., 2008. Data-based process monitoring, process control, and quality improvement: Recent developments and applications in steel industry. Comput. Chem. Eng. 32 (1–2), 12–24.

Khan, A., Dahl, M., Falkman, P., Fabian, M., 2018. Digital twin for legacy systems: Simulation model testing and validation. In: 2018 IEEE 14th International Conference on Automation Science and Engineering. CASE, pp. 421–426.

Kwak, S., Park, H., 2011. Adaptive control for interactive thermal processes. In: Proceedings of the 2011 IEEE International Conference on Control Applications. CCA, pp. 123–128.

Li, Y., 2017. Deep reinforcement learning: An overview. CoRR abs/1701.07274. arXiv:1701.07274. URL http://arxiv.org/abs/1701.07274.

Liu, M., Fang, S., Dong, H., Xu, C., 2021. Review of digital twin about concepts, technologies, and industrial applications. J. Manuf. Syst. 58, 346–361, Digital Twin towards Smart Manufacturing and Industry 4.0.

Ma, Y., Kassler, A., Ahmed, B.S., Krakhmalev, P., Thore, A., Toyser, A., Lindbäck, H., 2022. Using deep reinforcement learning for zero defect smart forging. In: The 10th Swedish Production Symposium 2022 (SPS2022). IOS Press, pp. 701–712. http://dx.doi.org/10.3233/ATDE220189.

Ma, Y., Younis, K., Ahmed, B.S., Kassler, A., Krakhmalev, P., Thore, A., Lindbäck, H., 2023. Automated and systematic digital twins testing for industrial processes. In: 2023 IEEE International Conference on Software Testing, Verification and Validation Workshops. ICSTW, pp. 149–158.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533.

Ogunfowora, O., Najjaran, H., 2023. Reinforcement and deep reinforcement learning-based solutions for machine maintenance planning, scheduling policies, and optimization. J. Manuf. Syst. 70, 244–263.

Panagou, S., Fruggiero, F., Lerra, M., del Vecchio, C., Menchetti, F., Piedimonte, L., Natale, O.R., Passariello, S., 2022. Feature investigation with digital twin for predictive maintenance following a machine learning approach. IFAC- Pap. 55 (2), 132–137, 14th IFAC Workshop on Intelligent Manufacturing Systems IMS 2022.

Pardo, N., Moya, J.A., 2013. Prospective scenarios on energy efficiency and CO2 emissions in the European iron & steel industry. Energy 54, 113–128. http://dx.doi.org/10.1016/j.energy.2013.03.015.

Pata, U.K., Kartal, M.T., Adebayo, T.S., Ullah, S., 2023. Enhancing environmental quality in the United States by linking biomass energy consumption and load capacity factor. Geosci. Front. 14 (3), 101531.

Perno, M., Hvam, L., Haug, A., 2022. Implementation of digital twins in the process industry: A systematic literature review of enablers and barriers. Comput. Ind. 134, 103558.

Pimenov, D.Y., Mia, M., Gupta, M.K., Machado, Á.R., Pintaude, G., Unune, D.R., Khanna, N., Khan, A.M., Tomaz, Í., Wojciechowski, S., et al., 2022. Resource saving by optimization and machining environments for sustainable manufacturing: A review and future prospects. Renew. Sustain. Energy Rev. 166, 112660.

Qin, J., Liu, Y., Grosvenor, R., 2016. A categorical framework of manufacturing for industry 4.0 and beyond. Procedia CIRP 52, 173–178, The Sixth International Conference on Changeable, Agile, Reconfigurable and Virtual Production (CARV2016).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction. The MIT Press.

Tao, F., Zhang, H., Liu, A., Nee, A.Y.C., 2019. Digital twin in industry: State-of-the-art. IEEE Trans. Ind. Inform. 15 (4), 2405–2415. http://dx.doi.org/10.1109/TII.2018.2873186.

Theorin, A., Bengtsson, K., Provost, J., Lieder, M., Johnsson, C., Lundholm, T., Lennartson, B., 2017. An event-driven manufacturing information system architecture for industry 4.0. Int. J. Prod. Res. 55 (5), 1297–1311. http://dx.doi.org/10.1080/00207543.2016.1201604.

Wang, J., Chortos, A., 2022. Control strategies for soft robot systems. Adv. Intell. Syst. 4 (5), 2100165.

Wang, J., Ma, Y., Zhang, L., Gao, R.X., Wu, D., 2018. Deep learning for smart manufacturing: Methods and applications. J. Manuf. Syst. 48, 144–156. http://dx.doi.org/10.1016/j.jmsy.2018.01.003.

Wang, S., Xu, X., 2011. Adaptive control of multivariable thermal processes in HVAC systems. Energy Convers. Manage. 52 (1), 462–470.

Zanoli, S.M., Cocchioni, F., Pepe, C., 2018. Model predictive control with horizons on-line adaptation: A steel industry case study. In: 2018 European Control Conference. ECC, IEEE, pp. 2332–2337.

Zhang, Q., Xu, J., Wang, Y., Hasanbeigi, A., Zhang, W., Lu, H., Arens, M., 2018. Comprehensive assessment of energy conservation and CO2 emissions mitigation in China's iron and steel industry based on dynamic material flows. Appl. Energy 209, 251–265. http://dx.doi.org/10.1016/j.apenergy.2017.10.084.

Zhao, Y., Belloum, A.S., Zhao, Z., et al., 2022. Mlops scaling machine learning lifecycle in an industrial setting. Int. J. Ind. Manuf. Eng. 16 (5), 138–148.

Dr. Bestoun S. Ahmed is a prominent researcher and professor in computer science, specializing in software engineering and quality assurance for AI systems. Dr. Ahmed completed his B.Sc. in Electrical and Electronics Engineering from Salahaddin University in Erbil, in 2004. He then pursued his M.Sc. in Computer Engineering from the University Putra Malaysia, graduating in 2009, and subsequently earned his Ph.D. in Software Engineering from Universiti Sains Malaysia in 2012. Dr. Ahmed's professional journey includes significant academic and research roles. He began as a lecturer at Salahaddin University, where he was promoted to Associate Professor in 2015. His postdoctoral research took him to the prestigious Swiss AI Lab, IDSIA, in Lugano, Switzerland. In 2016, he joined Czech Technical University in Prague as an Assistant Professor, where he founded the Software Testing Intelligent Lab (STILL) and served as the Red Hat software engineering chair. Since 2019, Dr. Ahmed has been part of Karlstad University in Sweden, initially as a senior lecturer and later promoted to Associate Professor. In 2023, he was appointed as a full Professor in the Department of Mathematics and Computer Science. His research at Karlstad focuses on software quality and digital modernization, particularly in machine learning systems, combinatorial testing, IoT systems, and applied soft computing.

Tommaso Azzalin is a skilled Data Engineer currently working at Uddeholm in Hagfors, Sweden. He has a robust educational background, having obtained his Master's degree in Computer Science from the University of Bologna and his Bachelor's degree from the University of Padua, both in Italy.

Dr. Andreas J. Kassler is a prominent computer science professor known for his extensive contributions to networking and cloud computing. He holds professorships at the Deggendorf Institute of Technology in Germany, where he leads the Intelligent Networks and Systems Lab, and Karlstad University in Sweden. Dr. Kassler joined Karlstad University in 2005 and has been integral to the institution's research in networking and cloud computing, focusing on areas such as Software Defined Networking, Future Internet, and Quality of Service. Dr. Kassler's academic journey includes a Ph.D. in Computer Science from the University of Ulm, Germany, obtained in 2002, and an M.Sc. in Mathematics and Computer Science from the University of Augsburg, Germany, earned in 1995. Prior to his current roles, he was an Assistant Professor at Nanyang Technological University in Singapore from 2003 to 2004.

Andreas Thore is a distinguished researcher in the field of industrial automation and artificial intelligence. He currently works as a researcher at the Research Institutes of Sweden (RISE) in Västerås, Sweden. His work primarily focuses on the application of AI in manufacturing processes, with a strong emphasis on quality assurance, process optimization, and digital twins. Dr. Thore has been involved in various high-impact projects such as the development of user-friendly AI systems for automated quality control in manufacturing, and the Smart Forge project, which aims to reduce the climate footprint of the forging industry through AI automation.

Hans Lindbäck is a prominent figure in the forging and steel industry, currently serving as a project manager at Bharat Forge Kilsta AB in Karlskoga, Sweden.